SerialPortManager

SerialPortManager User Manual

Version 0.3.0-rc.6 - Edition 2026-10-11

Copyright © 2026 Fation Coga. All rights reserved.

SerialPortManager is proprietary software of Fation Coga, licensed per PC (see LICENSE.md). This manual is part of the product: no part of it may be reproduced or distributed without the written permission of the copyright owner.

Contents

1. Introduction **5**

1.1 What you can do . . . 5

1.2 About this manual . . . 6

2. Installation and files **6**

2.1 Requirements . . . 6

2.2 Installing . . . 6

2.3 The data folder . . . 6

2.4 The manuals folder . . . 7

2.5 Uninstalling . . . 7

3. First start and trial **7**

4. The main window **7**

4.1 Tabs (slots) . . . 8

4.2 Menus . . . 9

4.3 Status bar . . . 9

4.4 Application log . . . 10

5. Serial > Serial Ports: how many ports, names and settings **10**

5.1 A port belongs to one tab . . . 10

6. Port settings **10**

6.1 Opening and closing . . . 11

6.2 Control lines and Break . . . 11

6.3 Lost ports and descriptions . . . 12

7. The Send box **12**

7.1 Notations . . . 12

7.2 Insert character table . . . 13

7.3 Counter / CRC... . . . 13

7.4 Commands and history . . . 14

8. The console **14**

8.1 Lines . . . 14

8.2 Console options . . . 15

8.3 Hint . . . 15

8.4 Receive options: framing and checksum check . . . 15

8.5 Find, filter and highlight . . . 16

8.6 Time between messages . . . 16

8.7 Statistics . . . 16

8.8 Capture, replay and export . . . 17

9. Auto replies and periodic messages **17**

Copyright © 2026 Fation Coga. All rights reserved.

9.1 Switches and lists . . . 17
9.2 Auto reply . . . 17
9.3 Periodic message . . . 18
9.4 The message editor . . . 18
9.5 Incrementing values . . . 18
9.6 Checksum . . . 19
10. Forwarding **20**
11. File transfer **20**
11.1 Sending a file . . . 20
11.2 Sending Options . . . 21
11.3 The two sending modes . . . 22
11.4 Catching files . . . 22
11.5 Catch options . . . 22
11.6 Recovery of an interrupted transfer . . . 23
12. Console right-click: analysis of a message **23**
13. The Byte Analyzer **25**
13.1 Checksums / CRC . . . 25
13.2 Integers / floats . . . 26
13.3 Signal . . . 27
13.4 Protocol . . . 27
14. The Protocols window **28**
14.1 Overview tab . . . 28
14.2 Protocol tabs . . . 29
14.3 Simulator parameters . . . 29
14.4 The waveform view . . . 31
15. The Modbus Master **32**
16. The Serial Plotter **33**
17. Test Sequences **33**
18. Profiles **34**
19. Licensing **35**
19.1 Trial, licensed and not licensed . . . 35
19.2 The License window . . . 35
19.3 Getting a license . . . 35
19.4 What a license checks . . . 36
20. Settings **36**
20.1 Theme . . . 36
20.2 Language . . . 36

Copyright © 2026 Fation Coga. All rights reserved.

20.3 Log level 37
20.4 Open Data Folder 37
20.5 Event Database 37
21. Help menu **37**
22. Troubleshooting **37**
23. Keyboard and mouse shortcuts **38**
24. Glossary **39**
25. Legal notices **40**

Copyright © 2026 Fation Coga. All rights reserved.

Version 0.3.0 - SerialPortManager (SerialPM) for Windows.

Copyright (C) 2026 Fation Coga. All rights reserved.

1. Introduction

SerialPortManager (SerialPM) is a Windows tool for working with serial ports: COM ports of the PC, USB-serial adapters, RS-232, RS-422 and RS-485 converters and any device that appears as a COM port. It shows everything received and sent, sends data typed in hexadecimal or text, answers received messages automatically, sends messages periodically, forwards data from one port to another, sends and receives whole files, and analyses bytes: checksums and CRCs, integers and floating-point numbers in every byte order, protocol frames (Modbus, NMEA) and the waveform the bytes produce on the wires.

Several ports can be controlled at the same time: every port has its own tab (a *slot*) with its own connection, background thread, console, automation and file transfer.

1.1 What you can do

- Open one or more serial ports, each with its own baud rate, data bits, parity, stop bits, flow control and timeouts.
- Send bytes written in **Hex**, **ASCII** or **Mixed** notation, with a table of all 256 characters.
- Read a console that shows every received (RX) and sent (TX) message with time, text and hexadecimal bytes, in colours by kind.
- Define **auto replies**: when certain bytes are received, reply with a message, optionally after a delay.
- Define **periodic messages**: send a message at a fixed interval.
- Put **incrementing values** (counters) and **checksums** (SUM, XOR, LRC, CRC-8/16/32) into replies and periodic messages.
- **Forward** everything a port receives to another port.
- **Send a file** in chunks, paced or confirmed chunk by chunk, with start prefix, end marker and recovery of interrupted transfers; **catch files** sent that way.
- Right-click any console message - or a selection of its bytes - for checksums, numbers, signal simulation and protocol decoding.
- Open the **Byte Analyzer** (about 50 checksum and CRC algorithms, any CRC from its parameters, every integer and float type, Modbus RTU / Modbus ASCII / NMEA 0183 decoding, waveform).
- Open the **Protocols** window: explanations and live simulations of UART, RS-232, RS-422, RS-485, SPI, I²C, 1-Wire, CAN, LIN, JTAG and the line codes (NRZ, Manchester...).
- Set the **RTS** and **DTR** outputs, watch **CTS**, **DSR**, **DCD** and **RI**, send a **Break**; have a lost USB port reopened when it comes back.
- Cut received bytes into messages by **pause**, **delimiter**, **fixed length**, **length field** or **Modbus RTU timing**, and check the **checksum / CRC** of every received message.
- **Find**, **filter** and **highlight** in the console, see the **time between messages** and the **statistics** of a port.
- **Capture** the traffic to a file, **replay** it, **export** the console as text or CSV; keep **saved commands** and the **history** of what you sent.
- Use the **Modbus Master**, the **Serial Plotter** and **Test Sequences** (send / expect scripts with pass / fail); move a whole setup with **profiles**.
- Work in English or Italian, with a light or dark theme.

1.2 About this manual

Labels of the user interface are written in **bold** exactly as the program shows them in English (the Italian manual quotes the Italian labels). Menu paths are written as **Menu > Item**. Keyboard shortcuts are written as Ctrl-B, F1 and so on.

The licensing system decides which features are available: see chapter 19. During the 14-day trial every feature is available.

2. Installation and files

2.1 Requirements

- Windows (64-bit or 32-bit build, as delivered).
- At least one serial port (COM port, USB-serial adapter or virtual port) to communicate with a device. Without ports the program still starts: the Byte Analyzer and the Protocols window work without any port.

2.2 Installing

SerialPortManager does not need an installer. Copy the delivered folder to any location where you can write, for example a folder in your documents, and start **SerialPortManager.exe**. Do not place it in a folder where Windows does not allow writing (for example *Program Files*) unless you run it with rights to write there: the program keeps its data next to the exe (see 2.3).

The delivered folder contains:

Item	What it is
SerialPortManager.exe	the program
DLL files	only in the builds that use them: wxWidgets, SQLite and serial libraries
SerialPortManager data	the data folder (created at the first start if missing)
manuals	the PDF manuals opened from the Help menu
THIRD-PARTY-NOTICES.md	the licences of the third-party components

2.3 The data folder

Everything the program writes goes into the folder **SerialPortManager data** next to the exe, so the whole state of an installation is one folder to back up, move or delete. If that folder cannot be created (read-only location), the folder of the exe is used instead. **Settings > Open Data Folder** opens it in Explorer, and **Help > Version Info...** shows its full path.

File	Content
AppSettings.txt	all settings: ports and their names, send box text, auto replies, periodic messages, file options, console options, theme, language, log level, simulator parameters
events.db	an SQLite database with one row per event: program start and close, license state, port opened / closed, every message received and sent (RX, TX, auto reply, periodic, forwarded) in hexadecimal, files sent and received
log.txt	the application log (the same lines as the Application log at the bottom of the window), appended at every start
language-it.xml	the Italian translation (one file per extra language: language-xx.xml)
license.lic	the installed license, if any
license.lic.bak	the previous license, after a new one was installed or the license was removed
license-request.txt	the license request of this PC, written automatically while the program is not licensed

Copyright © 2026 Fation Coga. All rights reserved.

File	Content
license-state.dat	the trial state (do not edit: an edited copy ends the trial)

The events database can be opened with any SQLite tool; its table **events** has the columns *time_utc*, *kind* (app, license, port, rx, tx, tx-auto, tx-periodic, tx-forward, file) and *text*. Received and sent bytes are prefixed with the title of their tab, for example `[COM3 - Meter] 01 03 00 00 00 01 84 0A`.

2.4 The manuals folder

Help > User Manual (F1) and **Help > Quick Guide** open the PDF files in the folder **manuals** next to the exe, in the language in use: SerialPM-Manual-EN.pdf / SerialPM-Manual-IT.pdf and SerialPM-QuickGuide-EN.pdf / SerialPM-QuickGuide-IT.pdf. They open in the default PDF viewer of Windows. If a file is missing, the program says so: *"The manual was not found or could not be opened: ... The manuals are installed in the "manuals" folder next to the program."*

2.5 Uninstalling

Close the program and delete its folder. To keep your settings and license for a later installation, keep a copy of the **SerialPortManager data** folder. The trial state is also kept in the Windows registry of the user (HKCU\Software\SerialPortManager\License); deleting files does not restart a trial.

3. First start and trial

At the first start on a PC:

1. The program creates the data folder and the log.
2. The English language and the light theme are used (change them in the **Settings** menu).
3. The 14-day trial starts: every feature is available. The status bar shows **Trial: N day(s) left**.
4. A license request (license-request.txt) is written in the data folder, ready to be sent to the license issuer.
5. The main window opens with one tab, **Slot 1**, without a port.

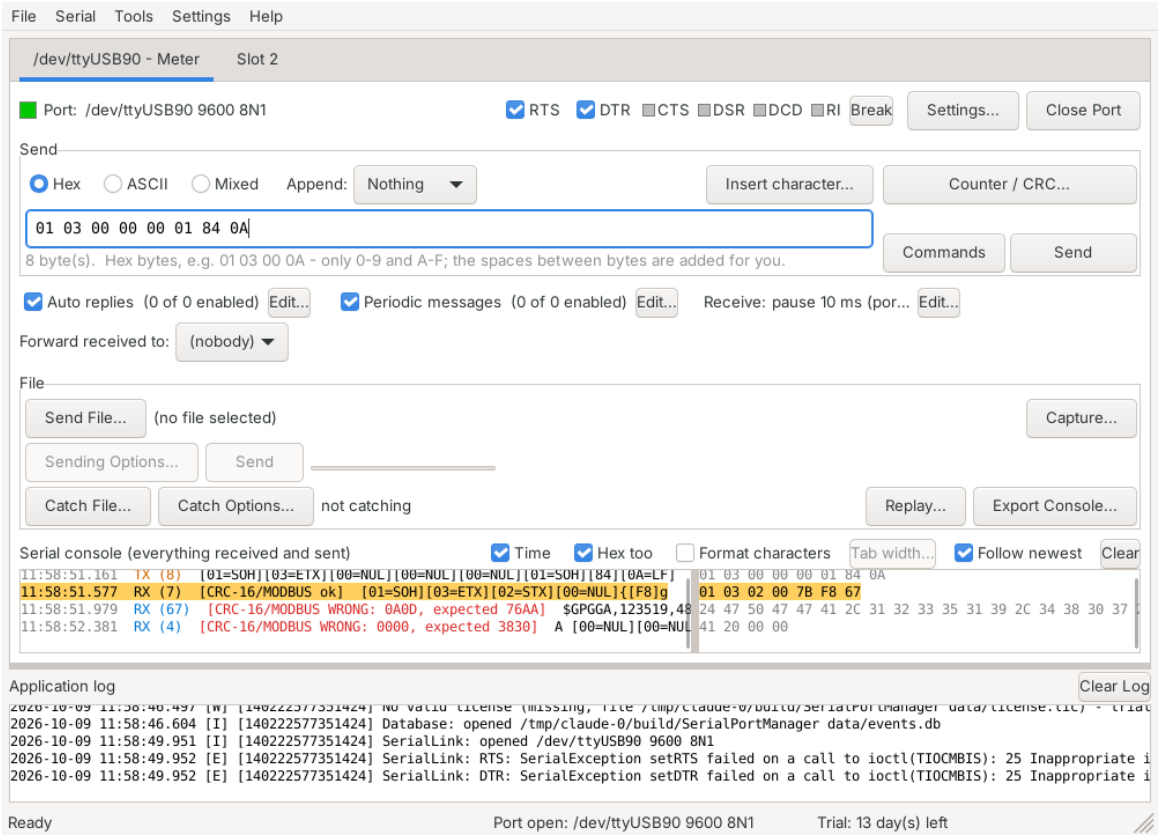
To start working:

1. Choose **Serial > Serial Ports...** to decide how many ports to control and to configure them, or click **Settings...** in the tab to configure only the port of that tab.
2. Click **Open Port**.
3. Type data in the **Send** box and press Enter or click **Send**.

The trial counts calendar days from the first start on that PC. After it, the program keeps working as a monitor (ports open, everything received is shown and logged); sending, auto replies, periodic messages, file transfer and forwarding need a license (chapter 19).

4. The main window

Copyright © 2026 Fation Coga. All rights reserved.



The main window with a port tab, the console and the application log

The title bar shows the program name, the version and the build date. From top to bottom the window has:

- the menu bar: **File**, **Serial**, **Tools**, **Settings**, **Help**;
- the port tabs, one per slot: each tab title is the port and the name you gave it, for example **COM3 - Meter** (only the name, or **Slot N**, while no port is set);
- under the tabs, separated by a bar you can drag, the **Application log** with its **Clear Log** button;
- the status bar.

4.1 Tabs (slots)

A slot is a tab with its own port, its own reading thread, its own console, its own auto replies and periodic messages, its own forwarding choice and its own file transfer. Slots work at the same time and independently. The number of slots and their names are set with **Serial > Serial Ports...** (chapter 5). The commands **Port Settings...**, **Auto Replies...**, **Periodic Messages...** and **Clear Console** of the **Serial** menu, and **Tools > Byte Analyzer...**, act on the tab that is selected.

Each tab contains, from top to bottom:

1. **The port row**: a status LED (green = port open, off = closed, orange = waiting for a lost port to come back), **Port:** followed by the port and its settings (for example *COM3 9600 8N1*, or **not set**; hovering shows what the device is, for example *USB-SERIAL CH340*), the control lines **RTS**, **DTR**, **CTS**, **DSR**, **DCD**, **RI** and **Break** (6.2), the **Settings...** button (port settings, chapter 6) and the **Open Port** / **Close Port** button.
2. **The Send box** (chapter 7), with **Commands** next to **Send** (7.4).

Copyright © 2026 Fation Coga. All rights reserved.

3. **The automation row**: the **Auto replies** and **Periodic messages** switches, each with a summary such as *(2 of 3 enabled)* and an **Edit...** button; **Receive:** with how received bytes become messages and its **Edit...** (8.4); **Forward received to:** (chapter 10). The row continues on a second line when the window is narrow.
4. **The File box** (chapter 11): **Send File...**, the file name, **Sending Options...**, **Send** (or **Stop**), a progress bar and the progress text; **Catch File...** (or **Stop Catching**), **Catch Options...** and the catch state; on the right **Capture...**, **Replay...** and **Export Console...** (8.8).
5. **The console**: **Serial console (everything received and sent)** in two panes (text and hex) with the options **Time**, **Hex too**, **Format characters**, **Tab width...**, **Follow newest** and the **Clear** button (chapter 8).

4.2 Menus

Menu item	Shortcut	What it does
File > Export Profile...		saves the ports' settings to a file (chapter 18)
File > Import Profile...		loads them from a file (chapter 18)
File > Exit	Alt-X	closes the program and its ports
Serial > Serial Ports...		number of ports, names, settings
Serial > Port Settings...		port of the current tab
Serial > Auto Replies...		auto replies of the tab
Serial > Periodic Messages...		periodic messages of the tab
Serial > Clear Console		empties the tab's console
Tools > Byte Analyzer...	Ctrl-B	analyses the send box bytes
Tools > Protocols (info and simulation)...	Ctrl-P	the Protocols window
Tools > Modbus Master...	Ctrl-M	the Modbus master window (chapter 15)
Tools > Serial Plotter...	Ctrl-L	the serial plotter (chapter 16)
Tools > Test Sequences...	Ctrl-T	test sequences (chapter 17)
Settings > Theme...		light or dark
Settings > Language...		English or Italian
Settings > Log Level...		how much the log records
Settings > Event Database...		limits, export and cleaning of events.db (20.5)
Settings > Open Data Folder		the data folder in Explorer
Help > User Manual	F1	this manual (PDF)
Help > Quick Guide		the short guide (PDF)
Help > License...	Ctrl-K	the License window
Help > Version Info...		version, build, history
Help > About...		name, copyright, libraries

4.3 Status bar

The status bar has three fields:

- left: general messages, for example **Ready** or **UID copied to the clipboard.**;
- middle: the port of the selected tab, **Port open: COM3 9600 8N1** or **Port closed**;
- right: the license state, **Licensed to NAME**, **Licensed to NAME - expires in N day(s)** (in the last 14 days), **Trial: N day(s) left**, or **Not licensed: sending, files and forwarding disabled (Help > License)**.

Copyright © 2026 Fation Coga. All rights reserved.

4.4 Application log

The lower part of the window shows the application log: start-up information, data folder, language loaded, license state, port problems (for example *the port cannot be read; retrying every second* when a USB adapter is unplugged) and other events, as *date time [level] [thread] message*. The view keeps the last 5000 lines; the complete history is in log.txt in the data folder. **Clear Log** empties the view only (not the file). How much is recorded is set with **Settings > Log Level...**.

5. Serial > Serial Ports: how many ports, names and settings

Serial > Serial Ports... opens the **Serial Ports** window:

- At the top, the ports found on this PC right now: **Serial ports on this PC: N (COM1, COM3, ...).** or **No serial port found on this PC right now.**
- **How many serial ports to control:** the number of slots (tabs), from 1 to the number shown in **(at most N)**. The maximum is the number of ports present on the PC (but never fewer than the slots that already exist), and never more than 32.
- The hint: **Every slot is a tab with its own port, thread and console. A port can belong to one slot only.**
- One row per slot: **Slot N**, **Name:** (free text; empty = the tab is called after its port or **Slot N**), the current port settings (or **not configured**) and **Configure...**, which opens the port settings dialog of chapter 6 for that slot.

Click **OK** to apply. Tabs are added or removed at the end; when the number is reduced, the last tabs are closed (their ports are closed). Reducing and increasing the number again inside the same dialog brings back the slots removed, with their settings. Ports that move from one slot to another are freed first, so a port can pass from one tab to another in one step.

5.1 A port belongs to one tab

A COM port can be opened by one program, and inside SerialPortManager by one slot only. In the port list of the settings dialog, a port already assigned to another slot is shown as **COM3 (already used by Slot 2 - Meter)**. If you choose it anyway, the program asks **Port already used**: *"COM3 is already used by Slot 2 - Meter. Detach it from that slot and use it here, or abort this choice?"*

- **Detach and use here**: the other slot is closed and loses the port (its console shows **Port detached** in red); this slot gets it.
- **Abort**: nothing changes; the port stays with the other slot.

6. Port settings

The **Serial Port** dialog opens from **Settings...** in a tab, from **Serial > Port Settings...** (current tab) or from **Configure...** in **Serial > Serial Ports...**.

Field	Values	Meaning
Port	list of the ports found, or typed	the COM port; **Refresh** reads the list again (for a USB adapter just plugged in)
Baud rate	1200 to 921600 from the list, or any value typed	bits per second; both sides must use the same
Data bits	5, 6, 7, 8	bits of data per character (8 is the usual)
Parity	None, Odd, Even, Mark, Space	the optional parity bit

Copyright © 2026 Fation Coga. All rights reserved.

Field	Values	Meaning
Stop bits	1, 1.5, 2	length of the stop bit(s)
Flow control	None, Software (XON/XOFF), Hardware (RTS/CTS)	how the receiver stops the sender
Inter-byte timeout (ms)	0 - 60000	silence that ends a received message (see below)
Read timeout (ms)	0 - 60000	longest wait of one read of the port
Write timeout (ms)	0 - 60000	longest wait for a write to complete
Reopen the port when it comes back (USB adapter unplugged)	on / off (on)	see 6.3

The default settings are 9600 baud, 8 data bits, no parity, 1 stop bit (8N1), no flow control, inter-byte timeout 10 ms, read and write timeouts 100 ms. A read or write timeout of 0 is raised to 50 ms.

How received bytes become messages. A serial line carries bytes, not messages. SerialPortManager groups the bytes received into one console line (one RX message) until the line stays quiet for the **Inter-byte timeout** (at least 5 ms), or until 4096 bytes have been collected. If your device's messages appear split over several RX lines, increase the inter-byte timeout; if several messages appear merged into one line, decrease it (it must be shorter than the pause between two messages of the device). Other ways of cutting the bytes into messages - a delimiter such as CR LF, a fixed length, a length carried in the message, Modbus RTU timing - are in the **Receive** options of the tab (8.4).

Write timeout. A long message at a low baud rate takes longer than the write timeout: the program writes it in slices that each take at most half of the write timeout, so long messages and file chunks are not cut. A write that still cannot complete (for example hardware flow control with the other side not ready) is reported in the console as **Write failed (...)**.

Clicking **OK** while the port is open closes it and opens it again with the new settings. If the port is now used by another slot, you are asked as described in 5.1.

6.1 Opening and closing

Open Port opens the port with its settings. The LED turns green, the button becomes **Close Port**, the middle field of the status bar shows **Port open: ...**, the send box becomes usable and the enabled auto replies and periodic messages start. If the port cannot be opened (used by another program, unplugged, wrong name) an error says **Cannot open COM3:** followed by the reason from Windows. **Open Port** is disabled while no port is set.

Close Port stops the reading thread, the periodic messages and a file being sent (it can be recovered later, chapter 11.6), and closes the port. Messages still waiting to be sent are dropped.

If a USB-serial adapter is unplugged while the port is open, the port is closed and opened again when it comes back (6.3).

6.2 Control lines and Break

The port row has the modem control lines of the port:

- **RTS** and **DTR** - the two outputs, ticked = active. Their state is remembered per tab and set every time the port opens; they can be changed while the port is closed. **RTS** is disabled with **Hardware (RTS/CTS)** flow control (the driver drives it then). Many boards reset when DTR changes (Arduino, ESP32); RTS often switches an RS-485 transceiver to transmit.
- **CTS**, **DSR**, **DCD**, **RI** - the four inputs, shown by LEDs (green = active), read ten times a second while the port is open.

Copyright © 2026 Fation Coga. All rights reserved.

- **Break** - holds the TX line in the break state (0) for 250 ms: a reset or a frame marker for some devices (LIN, DMX512, bootloaders).

Each change is noted in the console (label **Lines**), in red if the port or its driver refused it (virtual ports often have no modem lines).

6.3 Lost ports and descriptions

When the port stops working while it is open - typically a USB-serial adapter unplugged - the console says **COM3 lost (unplugged?) - closed; it is opened again when it comes back**, the LED turns orange and the button becomes **Stop Waiting**. Every second the program looks for the port; as soon as Windows lists it again it is opened with the same settings (**COM3 is back: opened again**), the automation and the control lines included. **Stop Waiting** gives up. The behaviour is set by **Reopen the port when it comes back (USB adapter unplugged)** in the port settings (on by default); with it off the program only retries reading every second.

The port list of the settings shows what each device is, as Windows names it, for example **COM3 (USB-SERIAL CH340)**, or its hardware id (*USB VID:PID=1A86:7523*); the port text of a tab shows it when hovered.

7. The Send box

The **Send** box of each tab holds the bytes to send by hand. Above the text are the notation buttons **Hex**, **ASCII** and **Mixed**, the **Append:** drop-down and the **Insert character...** button; under it a hint line shows the number of bytes and how to write them, or - in red - where the text is wrong and why. On the right, **Counter / CRC...** sets what Send inserts into the message (section 7.3), with a summary under it such as *+ counter 1 B, next 5 (last), CRC-16/MODBUS (last)*, and the **Send** button.

Press **Enter** in the text or click **Send** to send. The send box and **Send** are enabled only while the port is open (and **Send** is disabled while a file is being sent). If the text is not valid, a message such as **Character 7: only hex digits (0-9, A-F) are allowed.** is shown and the wrong character is selected. Sending needs the license feature *Sending*.

Append: adds **Nothing**, **CR**, **LF** or **CR and LF** at the end of every message sent. The text in the box stays as written; the console shows the bytes as they were sent, line end included.

The text, notation, **Append:** choice and the counter / CRC options of each tab are remembered when the program closes (the counter continues from its next value).

7.1 Notations

Notation	How to write	Example
Hex	pairs of hexadecimal digits; the spaces between bytes are added for you	`01 03 00 0A`
ASCII	printable ASCII characters only (space to ~)	`HELLO`
Mixed	text, and `//0x` followed by two hex digits for any byte	`OK//0x0D//0x0A`

Details:

- **Hex**: only 0-9 and A-F can be typed (lower case becomes upper case); the text is re-spaced while you type. Pasted C-style bytes such as `0x01, 0x03` are cleaned up: the `0x` prefixes and other characters are removed. A byte with one digit only is an error: **this byte has only one hex digit - a byte needs two.**
- **ASCII**: control bytes (CR, LF, 00...) cannot be written. Use Mixed for them.

Copyright © 2026 Fation Coga. All rights reserved.

- **Mixed**: one `//0x` for each byte, for example CR LF is `//0x0D//0x0A`. `//0x` must be followed by two hex digits: **//0x must be followed by two hex digits, e.g. //0x0D.** A literal text "//0x" in the data is written as `//0x2F/0x...`: the program does this for you when converting.

Clicking another notation converts the current text: the bytes stay the same. If you switch to ASCII and the data contains bytes ASCII cannot show, the program asks whether to switch to Mixed instead (**No** keeps the current notation).

7.2 Insert character table

Insert character... opens the **Insert Character** table with all 256 byte values: **Dec**, **Hex**, **Char**, **Name** (for example NUL, STX, CR, LF, ESC, DEL) and **Description** (for example *Carriage return*, *Capital letter A*, *Not ASCII (shown as its Latin-1 character)* for 80-FF).

- Double-click a character to add it at the cursor and close the table.
- Right-click it and choose **Add X and keep this table open**, or select it and click **Add, keep open**, to add several characters; the line **Added: ...** shows what was added.
- **Close** closes the table.

The character is written in the notation in use: in Hex as two digits, in Mixed as itself or as `//0xHH`; in ASCII, adding a control byte switches the text to Mixed first.

7.3 Counter / CRC...

Counter / CRC... opens the **Counter and checksum** dialog. What it sets is INSERTED by Send into the typed message; the text in the box never changes.

Counter - **Insert an incrementing counter**:

- **Bytes**: 1 to 8 (default 1).
- **Next value**: the value the next message carries (decimal, `0x` hex or `0b` binary).
- **Added per send**: added after every message sent; negative counts down. Past the largest value of its bytes it starts again from 0 (255 + 1 = 0 with one byte).
- **Byte order** (with more than one byte): **Little-endian (least significant byte first)** - the default - or **Big-endian (most significant byte first)**.
- **Position** and **Bytes away**: **Counted from the end** - 0 = last, 1 = before the last byte, 2 = before the last two... - or **Counted from the start** - 0 = first, 1 = after the first byte...

Checksum / CRC - **Insert a checksum / CRC of the message**:

- **Algorithm**: every checksum and CRC of the Byte Analyzer (sums, XOR, LRC, Fletcher, Adler, CRC-8 ... CRC-64). Choosing one sets the byte order it is usually sent in (for example CRC-16/MODBUS little-endian).
- **Byte order**, **Position**, **Bytes away**: as for the counter.
- **Include the counter in the calculation**: whether the counter's bytes are covered. When the counter and the checksum are both switched on, the program asks *Include the counter in the checksum / CRC calculation?*

The message is built in this order: the counter is inserted, the checksum is calculated (over the typed bytes, with or without the counter) and inserted, then the **Append:** line end is added - the line end is never part of the checksum.

The **Next message** box previews the next message with the current text and line end: the counter in `[ ]`, the checksum in `{ }`, for example `01 03 00 00 00 01 [07] {4B A1} 0D 0A`, and the counter's

Copyright © 2026 Fation Coga. All rights reserved.

value after it.

Example - a Modbus RTU request with its CRC: text `01 03 00 00 00 01`, checksum CRC-16/MODBUS, last, little-endian: Send writes `01 03 00 00 00 01 84 0A`.

The same data entry (notations, hint, character table) is used everywhere the program asks for bytes: auto reply patterns and replies, periodic messages, file commands, the Byte Analyzer and the Protocols window.

7.4 Commands and history

Up and **Down** in the send text bring back the messages sent from this tab, newest first (the last 50, kept between sessions), with their notation and **Append:**; **Down** past the newest returns to what you were typing. Sending a message again moves it to the front.

Commands, next to **Send**, opens a menu:

- the **saved commands**: a click loads one into the send box and sends it (the counter / CRC options of the tab are applied as for **Send**);
- **Save the text as a command...** - asks for a name (the same name replaces a command);
- **Delete command** - a submenu of the saved commands;
- **Sent before** - the history: a click loads a message without sending it;
- **Clear the history**.

The saved commands are shared by all tabs.

8. The console

The console of each tab, **Serial console (everything received and sent)**, has two panes: the messages as text on the left and, with **Hex too**, the same bytes in hexadecimal on the right - line for line, every hex line beside the text line it belongs to. The two panes scroll together (scrolling either one moves the other) and the divider between them can be dragged (its position is remembered). The console keeps the last 5000 lines, oldest at the top.

8.1 Lines

A message line has the form:

```
14:03:21.512  RX (6)  [06=ACK]OK 1[0D=CR]        06 4F 4B 20 31 0D
```

- the time with milliseconds (option **Time**);
- the label and, in brackets, the number of bytes;
- the bytes as text: printable characters as themselves, control bytes as `[HH=NAME]` (for example `[0D=CR]`), other bytes (above 7E) as `[HH]`;
- in the right pane, the bytes of this line in hexadecimal (option **Hex too**).

With **Format characters** on, LF, CR, CR and LF, vertical tab (0B) and form feed (0C) start a new line: a message of several text lines is shown on several console lines, the following ones under the start of the first (without the time and label again), each with its own bytes in the hex pane (the line end included). A tab (09) becomes spaces. Every other control character stays visible as `[HH=NAME]`, every byte above 7E as `[HH]`.

Label	Colour shown	Meaning
RX	blue	received

Copyright © 2026 Fation Coga. All rights reserved.

Label	Colour shown	Meaning
TX	orange	sent from the send box
TX auto "name"	purple	sent by the auto reply *name*
TX periodic "name"	green	sent by the periodic message *name*
TX fwd "tab"	teal	received by the tab *tab* and forwarded through this port
TX file prefix	amber	start prefix of a file (chapter 11)
TX file end	amber	end marker of a file
TX file recovery	amber	recovery command of a file
TX file answer	amber	answer sent while catching a file
File selected	amber	a file was chosen
Sending file	amber	a file transfer starts; each chunk is a **TX** line such as *chunk 3/10 (64 bytes)*
Recovering file	amber	an interrupted transfer continues
File	amber	file transfer results and catch notes
TX replay	green	sent by Replay (8.8)
TX Modbus / TX Sequence	teal	sent by the Modbus master (chapter 15) or a test sequence (chapter 17)
Lines	teal	a control line changed or a break was sent (6.2)
Port	red / teal	the port was lost / came back (6.3)
Capture, Replay, Export	teal / green	capture, replay and export notes (8.8)
any label	red	an error: **Write failed (...)**, **Too many messages waiting - this one was not sent.**, an auto reply or periodic message that cannot be built, **Port detached**, a forwarding that could not be done, a failed file transfer

The red note **Auto replies and periodic messages not started: they need a license (Help > License).** appears when the switches are on but the license does not include those features.

8.2 Console options

- **Time**: show the time at the start of each line.
- **Hex too**: show the hex pane (it is always filled: switching it on shows the earlier lines too).
- **Format characters**: line ends start a new line and tabs become spaces (see above). Switching it on asks how many spaces a tab takes; **Tab width...** changes it (1 to 16).
- **Follow newest**: keep the newest line in view - in both panes, since they scroll together. Turn it off to read older lines while data keeps arriving.
- **Clear**: empties both panes (also **Serial > Clear Console** and the right-click menu).

The options are common to all tabs and are remembered; **Time** and **Format characters** apply to the lines written after the change.

8.3 Hint

Hovering over the console shows the hint **Right-click a message - or select some of its bytes, in the text or in the hex part - for checksums, numbers, signal simulation and protocol decoding.** The right-click menu is described in chapter 12.

8.4 Receive options: framing and checksum check

Receive: in the automation row shows how the received bytes become messages (for example *pause 10 ms (port)*, *ends with 0D 0A, CRC-16/MODBUS*); **Edit...** opens the **Receive Options**.

Messages end:

Copyright © 2026 Fation Coga. All rights reserved.

Choice	A message ends	Settings
after a pause (gap)	when no byte arrives for the gap	**Gap (ms):** 0 = the port's inter-byte timeout
with a delimiter (e.g. CR LF)	with the delimiter bytes	**Delimiter (hex):** for example `0D 0A`, `0A`, `03`; **Show the delimiter**
after a fixed number of bytes	every N bytes	**Bytes per message:** 1 - 4096
after the length carried in the message	at offset + field + value + adjust bytes	**Length at byte:**, **Length bytes:** 1 - 4, **Byte order**, **Adjust:**
Modbus RTU (3.5 characters of silence)	after 3.5 character times of silence at the port's speed (1.75 ms above 19200 bit/s)	-

Except the gap, a message that stays incomplete is shown anyway after **Timeout (ms):**, so nothing is lost when the device does not follow the framing. Any message is shown at the latest at 4096 bytes. Example - frames *STX, length (2 bytes, little-endian), data, a 1-byte checksum*: **Length at byte** 1, **Length bytes** 2, little-endian, **Adjust** +1. Through USB adapters the Modbus RTU timing is only as good as the adapter's latency: if frames come split or joined, use a gap of a few ms.

Check received messages - **Check the checksum / CRC of every received message**: the **Algorithm** (every one of the Byte Analyzer), its **Byte order**, its **Position** (**from the end**: **Offset** = the bytes after it, 0 = last, 2 = a CR LF follows; **from the start**: the byte where it begins) and **Not covered at the start:** (e.g. 1 for an STX that is not part of it). The checksum covers the bytes before it. Every RX line then shows the result right after its label: **[CRC-16/MODBUS ok]** in green, or in red **[CRC-16/MODBUS WRONG: 0A84, expected 0B94]** or **[too short for CRC-16/MODBUS]**.

8.5 Find, filter and highlight

Ctrl-F in the console (or **Find...** in its right-click menu) opens the find bar under the console options: type the text; **Enter** / **>** finds the next match, **Shift-Enter** / **<** the previous one, **F3** / **Shift-F3** in the console do the same; the bar says *3 of 12* or *not found*. **Match case**; **In hex** looks in the hex pane (for example `0D 0A`); **Only matching lines** shows only the lines containing the text. **Esc** or **x** closes the bar (and ends that filter).

The right-click menu **Show** switches **Received (RX)**, **Sent (TX, auto replies, periodic, forwarded, replay)** and **Notes (port, files, capture...)** on and off (per tab, remembered). While something is hidden the console title says **Serial console - filtered: 12 of 40 lines**. The hidden lines are kept: switching them on again shows them in place.

Highlight rules... (right-click menu): messages that contain a text (case ignored, e.g. *ERROR*) or a byte sequence (**Hex bytes**, e.g. `01 83`) are shown on a colour, in both panes and in every tab. **Add**, **Replace**, **Delete**, **Move Up**; the tick switches a rule on and off; the first ticked rule that matches wins. The text is black or white, whichever reads on the colour.

8.6 Time between messages

Time since the previous message (delta) in the right-click menu adds, after the time, the seconds since the previous message of the tab, for example `14:03:21.512 +0.302 RX (8) ...` - to measure response times and periods.

8.7 Statistics

Statistics... (right-click menu) shows, updated twice a second: **Counting since**, **Port open for**, **Received** and **Sent** (bytes and messages), **Receiving now** / **Sending now** (bytes per second), **Write errors** and **Checksum of received** (ok, wrong, too short - with the check of 8.4). **Reset** starts counting again. File data being caught is not counted as received; framing, parity and overrun errors of the line are not reported by the serial library.

Copyright © 2026 Fation Coga. All rights reserved.

8.8 Capture, replay and export

Capture... (File box) writes the traffic of the tab to a file while it happens, until **Stop Capture**:

- **Timed capture (*.spmcap)**: every message received and sent, with its time - for Replay;
- **Raw received bytes (*.bin)**: the received bytes one after the other - for other tools.

The size captured is shown next to the button; the console notes the start and the end.

Replay... sends the messages of a timed capture again, with their pauses: **what was sent (repeat a session)** or **what was received (play the other device)**, at **1x**, **2x**, **5x**, **10x** or **no pauses** (pauses longer than 10 s are shortened to 10 s). The messages appear as **TX replay**; **Stop Replay** ends it. Replay needs the license feature *Sending*.

Export Console... (also in the right-click menu) saves the console as shown (a filter applies): **Text (*.txt)**, **the lines as they are, or** CSV (*.csv)*, one row per message with *time, label, bytes, hex, text, note* (UTF-8 for spreadsheets).

9. Auto replies and periodic messages

9.1 Switches and lists

Each tab has two switches in the automation row:

- **Auto replies**: when ticked, the enabled auto replies of this tab are active while the port is open.
- **Periodic messages**: when ticked, the enabled periodic messages of this tab are sent while the port is open.

Next to each switch, **(N of M enabled)** tells how many items are enabled, and **Edit...** opens the list. Ticking a switch without the needed license feature asks whether to open the License window.

Serial > Auto Replies..., **Serial > Periodic Messages...** and the **Edit...** buttons open the window **Auto Replies and Periodic Messages**, with two tabs:

- **Auto replies**: *While the port is open, each ticked rule replies as soon as its bytes are received. Double-click to edit.* Columns **Name**, **When received**, **Reply**, **Delay (ms)**.
- **Periodic messages**: *While the port is open, each ticked message is sent at its interval. Double-click to edit.* Columns **Name**, **Message**, **Every (ms)**.

The tick box at the start of each row enables or disables the item. The buttons on the right: **Add...**, **Edit...** (or double-click), **Duplicate** (a copy below the selected item) and **Remove** (asks **Remove "name"?**). Changes take effect when you click **OK** (and are discarded with **Cancel**); they are saved with the tab.

9.2 Auto reply

The **Auto Reply** dialog has:

- **Name:** (default *Auto reply N*) and **Enabled**;
- **When these bytes are received**: the pattern, in Hex, ASCII or Mixed. It must not be empty (**Write the bytes to recognise.**);
- **Reply with**: the message editor (9.4);
- **Delay before replying (ms):** 0 to 600000 ms; 0 replies at once.

How it works:

Copyright © 2026 Fation Coga. All rights reserved.

- The pattern is searched in the stream of received bytes, also when it arrives split across several reads or in the middle of a longer message. Every occurrence triggers the reply, and several rules can reply to the same bytes.
- With no delay the RX line containing the pattern is shown first, then the reply; with a delay, the reply is sent when the delay has passed (other messages can be sent and received meanwhile).
- The reply is shown as **TX auto "name"** in purple.
- Auto replies pause while this tab is sending a file (the confirmations of the receiver must not trigger them) and do not see the bytes of a file being caught.
- Incrementing values of a reply start again from their start value when the port is opened or the list is changed.
- A reply that cannot be built (for example an invalid text saved by hand) is reported in red: **Auto reply "name" cannot be built - check its text.**

9.3 Periodic message

The **Periodic Message** dialog has **Name:** (default *Periodic message N*), **Enabled**, **Send every (ms):** (10 to 86400000 ms, that is up to 24 hours) and the **Message** editor (9.4).

Each enabled message has its own timer; the first message is sent one interval after the port opens (or after the list is changed). It is shown as **TX periodic "name"** in green. While a file is being sent, periodic messages are skipped (they would end up in the middle of the file). A message that cannot be built is stopped and reported once: **Periodic message "name" cannot be built - check its text. It has been stopped.**

If more than 1000 messages are waiting to be written (for example very short periods at a low baud rate), new ones are refused with **Too many messages waiting - this one was not sent.**

9.4 The message editor

Replies, periodic messages and the commands and answers of file transfer are edited with the same message editor:

1. The fixed bytes, in Hex, ASCII or Mixed (with **Insert character...**).
2. **Incrementing values (written over the message bytes):** a list of counters with **Add...**, **Edit...** (or double-click) and **Remove**. Each line describes the counter, for example *Bytes 2-3 (Binary number, big-endian): 0 to 65535, step 1, 1 step(s) per message sent*.
3. **Checksum:** a summary (for example *CRC-16/MODBUS, 2 byte(s) at the end, little-endian*, or *none*) and the **Checksum...** button.
4. The hint: *an incrementing value is written OVER the fixed bytes at its index - if the fixed text has a byte there, it is replaced, so put a placeholder (a "don't care" byte) at that position. The checksum is INSERTED at its index and moves the following bytes.*
5. **Preview of the first messages (one second apart):** the first four messages as they will be sent, for example `#1 (8 bytes): 01 03 00 00 00 01 84 0A | ...`, so counters advancing per second are visible too.

The message is built in this order: fixed bytes, then the incrementing values written over them, then the checksum computed and inserted. An empty message is refused: **Write the message to send.**

9.5 Incrementing values

The **Incrementing Value** dialog:

Copyright © 2026 Fation Coga. All rights reserved.

Field	Meaning
Written as:	**Binary number** (1-8 bytes), **ASCII decimal digits** (1-20 characters, with leading zeros) or **ASCII hex digits** (1-16 characters, upper case)
Size:	number of bytes (the bits are shown: *bytes (16 bits)*) or **characters**
Starts at byte index:	the first byte it occupies (0 = first byte of the message); the line below says which bytes it covers, and if the message must be extended with zero bytes
Byte order:	**Big-endian (most significant byte first)** or **Little-endian (least significant byte first)**, for binary values of 2 bytes or more
Start value:, End value:	decimal, or hex as 0x1F
Step:	positive, or negative to count down (from a start value above the end value)
Advances:	a number of **step(s) per message sent** or **per second**; fractions are allowed: 0.5 per message = one step every 2 messages

Big-endian is the default: Modbus and most binary protocols send numbers this way. The values go from the start value by the step, never beyond the end value, then start again from the start value. The information lines show the first values, the rate (for example *one step every 2 messages sent*) and **Largest value for this size: ...**. Errors are explained (size out of range, values not fitting the size, step 0, step in the wrong direction, number of steps not greater than 0).

For an answer the program **expects** (file transfer, chapter 11), an incrementing value advances by one step for every answer expected, so answers such as *ACK 01*, *ACK 02*... can be recognised.

9.6 Checksum

The **Checksum** dialog:

- **Add a checksum to the message**: switches the checksum on.
- **Also cover the message this one answers: ...** (only for answers, see below), with **Start it with the file's start prefix (then the chunk / group)** (file confirmations and group answers only) and **Leave out of it the indexes:** (for example `0-2, -1`; -1 = the last byte).
- **Type:** one of the 12 types below, shown as *name - N byte(s)*. Choosing a type also selects its usual byte order.
- **Insert at index:** where the checksum goes (0 = before the first byte, N = after the last one), or **at the end (follows the length of the message)**.
- **Calculated on: all the bytes** or **all the bytes except the indexes:** (for example `0, 5-7`; 0 = first byte) - for example to leave out a start character.
- **Bytes to send:** how many bytes of the checksum are sent (**of N**), and for fewer: **lower part (least significant bytes)** or **upper part (most significant bytes)**.
- **Byte order:** big-endian or little-endian.
- The preview: **Checksum of the current message: 0x.... - sent as .. .. at index N.** and the whole message.

Type	Bytes	Usual order
SUM-8 (add)	1	-
SUM-16 (add)	2	big-endian
XOR-8 (BCC)	1	-
LRC-8 (two's complement of the sum)	1	-
CRC-8 (SMBus)	1	-
CRC-8/MAXIM (1-Wire)	1	-
CRC-16/MODBUS	2	little-endian
CRC-16/ARC (IBM)	2	little-endian

Copyright © 2026 Fation Coga. All rights reserved.

Type	Bytes	Usual order
CRC-16/CCITT-FALSE	2	big-endian
CRC-16/XMODEM	2	big-endian
CRC-32 (IEEE, zip)	4	little-endian
CRC-32C (Castagnoli)	4	little-endian

The checksum is calculated on the message after the incrementing values have been written, without the excluded indexes, and is recalculated for every message sent.

Covering the answered message. Some devices compute the checksum of an answer over the request too. For answers, the checksum can also cover *the message this one answers*: its bytes are taken first (without the indexes you leave out; negative indexes count from the end, and an index a shorter message does not have is ignored), then the answer's own bytes (**Calculated on** applies to the answer only). What "the answered message" is depends on where the editor is:

Editor	The answered message
auto reply	the received RX line, up to the end of the pattern
file send: prefix answer	the start prefix sent
file send: confirmation	the chunk sent (its data bytes)
file send: recovery answer	the recovery command sent
catch: prefix answer	the start prefix received
catch: group answer	the group of file bytes received
catch: recovery answer	the recovery command received

The preview then says **(preview without the answered message: it is known only when it arrives)**. Periodic messages answer nothing, so they do not have this option.

10. Forwarding

Forward received to: on the right of the automation row chooses another tab: everything this port receives is sent out of that tab's port. **(nobody)** switches forwarding off. The list shows the other tabs by title and is disabled when there is only one tab.

- Forwarded data appears in the target tab as **TX fwd "source tab"** in teal; the source tab shows it as normal RX.
- If the target port is closed, the source console shows in red **not forwarded: the port of TAB is closed**.
- If the target tab is sending a file, or too many messages are waiting, the target console shows the data in red with **Too many messages waiting - this one was not sent.**
- Forwarding goes one way. To connect two ports in both directions (a "bridge" between two devices, with SerialPortManager listening in the middle), set each tab to forward to the other.
- Forwarding needs the license feature *Forwarding to another port*; if the license loses it, forwarding is set back to (nobody).

11. File transfer

The **File** box of each tab sends a file over the port and catches files sent the same way, for example to load firmware or data into a device or to receive logs from it. File transfer needs the license feature *File transfer*.

11.1 Sending a file

Copyright © 2026 Fation Coga. All rights reserved.

1. **Send File...** chooses the file (up to 256 MB). Its name and size appear next to the button and the console shows **File selected**.
2. **Sending Options...** sets how it is sent (11.2).
3. With the port open, **Send** starts the transfer. The button becomes **Stop**, the bar shows the progress and the text shows *sent / total*. Before starting, the text shows the minimum time, **at least X**; at the end, **sent in X**.
4. **Stop** cancels the transfer.

While a file is being sent the **Send** button of the send box is disabled, periodic messages are skipped, auto replies pause and data forwarded to this tab is refused.

The console shows **Sending file** (or **Recovering file**), **TX file prefix**, one line per chunk (**TX** *chunk 3/10 (64 bytes)*, with *(retry)* when a chunk is sent again; the bytes of the chunks are not printed), **TX file end**, and at the end **File** *file sent: SIZE in TIME*, or *file sending failed after X of Y: reason*, or *file sending stopped after X of Y*.

11.2 Sending Options

The **Sending Options** window has seven tabs.

Chunks

- **Bytes per chunk:** 1 to 1048576 (default 64). *The file is sent in chunks of this size; the last chunk holds the remaining bytes and can be smaller.*
- **Every chunk, then pause** N **ms** (0 - 600000; 0 = as fast as the port goes): paced mode.
- **Burst: after every chunk wait for the confirmation (tab Confirmation)**: confirmed mode.
- **Answer timeout (ms):** (1 - 600000, default 1000) and **Retries:** (0 - 1000, default 3): how long an expected answer may take, and how often the chunk (or the prefix, or the recovery command) is sent again without an answer. *If only the confirmation was lost, the receiver gets that chunk twice* (recovery, 11.6, avoids this).
- The estimate, for example **12 chunk(s) at 115200 baud: at least 0.07 s (answer times of the other side not included)**. It counts every byte on the line (data, prefix, end marker, the expected answers in burst mode) at the port's speed, plus the pauses. Without a file: **Select a file to see the estimated time.**

Start prefix - **Send this before the file (signals that a file follows)**: a message sent once before the first chunk.

Prefix answer - **Wait for this answer before sending the file**: the answer expected from the other side after the prefix. *Expected from the other side; incrementing values and checksum are compared too.* Without it in time, the prefix is sent again (Retries).

Recovery - **Allow recovery: continue an interrupted file where the receiver stopped** (11.6).

Recovery answer - expected after the recovery command, followed by 4 bytes (big-endian): how many bytes of the file the receiver already has.

Confirmation - burst mode: the answer expected after every chunk before the next one is sent. An incrementing value advances by its step for every confirmation.

End marker - **Send this after the last chunk**.

Every command and answer tab has **Use the recommended value: ...**: ticking it fills the editor with the recommended value and locks it. The recommended values are 8-byte commands very unlikely to occur inside a file, the same on both sides:

Copyright © 2026 Fation Coga. All rights reserved.

Command	Recommended value
Start prefix	A5 5A 02 53 50 4D 46 01
Prefix answer	A5 5A 06 53 50 4D 46 01
Recovery	A5 5A 02 53 50 4D 46 12
Recovery answer	A5 5A 06 53 50 4D 46 12 (+ 4 bytes)
Confirmation	A5 06 + CRC-16/MODBUS of the chunk
End marker	A5 5A 04 53 50 4D 46 04

The recommended confirmation carries a CRC-16/MODBUS of the chunk it confirms (the checksum *covers the answered message*), so it is also an integrity check of each chunk.

OK checks the settings: every command in use must be valid and not empty (*Start prefix: the pattern is empty.*), and the commands sent - like the answers expected - must be different and neither may contain another (**"Start prefix" and "End marker" must be different, and neither may contain the other: the other side could mistake one for the other.**). The tab with the problem is shown.

11.3 The two sending modes

- **Paced** (*Every chunk, then pause*): chunk, pause, chunk... The receiver must keep up by itself. Use a pause long enough for a device that writes each chunk to memory.
- **Burst** (*after every chunk wait for the confirmation*): chunk, wait for the confirmation, next chunk. Without a confirmation within the answer timeout the same chunk is sent again, up to Retries times; then the transfer fails with a reason that says which answer was expected, for example *no confirmation for chunk 4 (expected: A5 06 ...)*.

11.4 Catching files

Catch File... opens the **Catch File** options (11.5); **OK** arms catching. The button becomes **Stop Catching**, the state text says **waiting for a file...** and the console shows **File** *catching files into PATH*. Catching can be armed with the port closed: it starts when the port opens.

- Bytes received before the start prefix are shown and handled as normal RX (auto replies included).
- When the start prefix arrives, a file starts: the console shows *file start recognised: PATH is written again from the beginning*, and the state text shows **receiving: SIZE**. From then on the bytes are file data: they are written to the output file and not shown in the console.
- The answers the catcher sends appear as **TX file answer**.
- The file ends with the end marker or after the silence time; the console shows *file received: SIZE in PATH* and the state **N file(s) received - waiting for a file...**. The catcher then waits for the next file.
- **Stop Catching** disarms it (*file catching stopped*).

Catch Options... changes the options also while catching: a file being received ends with the previous options, the next one uses the new ones (the console says *catch options changed (a file being received ends with the previous ones); writing to PATH*).

11.5 Catch options

File

- **Write the received file data to:** the output file, with **Browse...**. *Only the file data is written (no prefix, answers or end marker). A new file replaces what the output file held; a recovered file (Recovery tab) continues it.*
- **Bytes per group (answered each):** 1 to 1048576 (default 64).

Copyright © 2026 Fation Coga. All rights reserved.

- **File ends after a silence of (ms, 0 = never):** 0 to 3600000 (default 2000).

Start prefix - *A file starts when this is received. The bytes before it are shown as normal RX.* Required.

Prefix answer - **Answer the prefix with this**.

Recovery - **Accept recovery: an interrupted file continues (the output file is kept)**.

Recovery answer - *Sent after the recovery command, followed by the size the output file has (4 bytes, big-endian).*

Group answer - **Answer every group of bytes with this**: *Sent after every group; a smaller last group is answered when the line goes quiet. An incrementing value advances for every answer. If its checksum covers the group, use the sender's chunk size as the group size, so each answer covers exactly one chunk.*

End marker - **The file ends with this marker**: *The marker is not written to the file.*

OK refuses an empty output file (**Choose the file where the data is written.**), a file that would never end (**A file would never end: use an end marker or a silence time.**) and commands that clash, as for sending.

To transfer between two SerialPortManager installations, use the same commands on both sides: tick **Use the recommended value** everywhere, set the sender's **Bytes per chunk** equal to the catcher's **Bytes per group**, and choose burst mode on the sender with the group answer on the catcher.

11.6 Recovery of an interrupted transfer

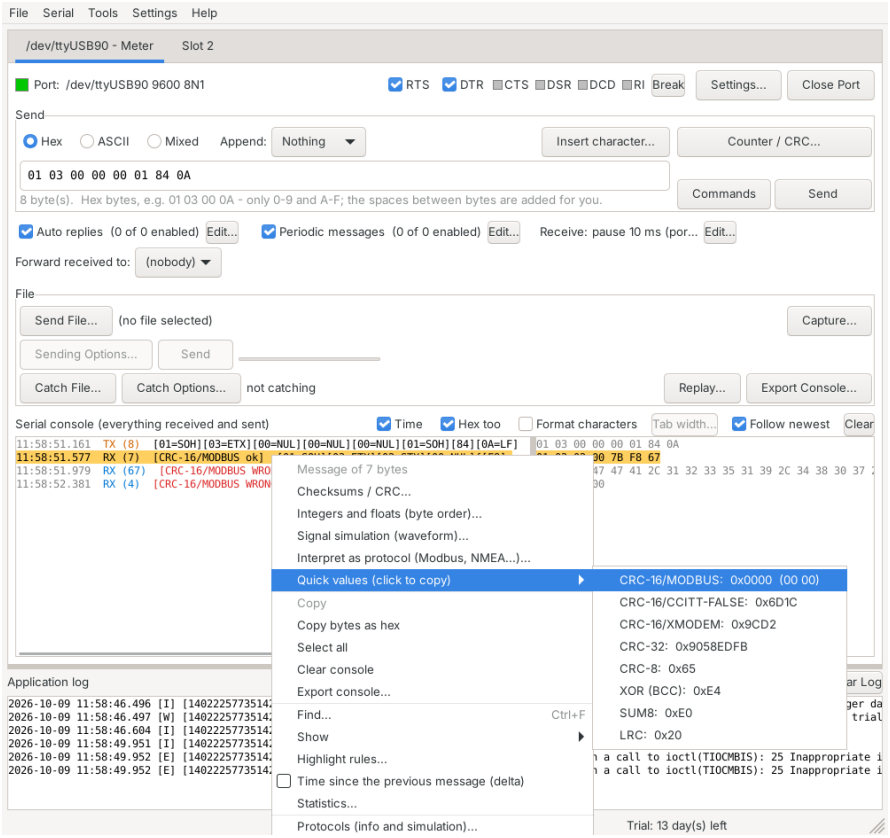
When a send stops part-way (Stop, a failed confirmation, the port closed, the program closed), the program remembers the file and how far it got. If recovery is allowed in the sending options and you send the same file again (same path and size), it asks **Recover the file?**: *The last sending of NAME stopped after X of Y (confirmed).*

- **Recover**: the recovery command is sent instead of the start prefix; the receiver keeps what it has and answers with the recovery answer followed by its file size (4 bytes, big-endian); sending continues from that byte. The console shows **TX file recovery** and *the receiver has X of Y: continuing from there*. Because the receiver's size is the truth, a chunk whose confirmation was lost is neither sent twice nor skipped.
- **Start Again**: the file is sent from the beginning and the receiver starts it again.
- **Cancel**: nothing is sent.

On the catching side, **Accept recovery** keeps the output file when the recovery command arrives - instead of the start prefix, or in the middle of a file that did not end - answers with its size and appends the data that follows (*recovery: X already in PATH, continuing it*). A start prefix or recovery command repeated before any data has arrived (because the sender did not get the answer) is answered again instead of being taken as file data.

12. Console right-click: analysis of a message

Copyright © 2026 Fation Coga. All rights reserved.



The right-click menu of the console

Right-click a message line of the console to analyse its bytes. You can also select part of a message first - in the text pane or in the hex pane, even across several lines - and right-click inside the selection: then only the selected bytes are used (the lines of one message add up to one part of it; the bytes of several messages are joined in order). With the keyboard (context-menu key) the menu uses the selection.

The menu contains:

- a header (not clickable): **Message of N bytes** for a whole message, or **N selected bytes** for a selection;
- **Checksums / CRC...** - opens a Byte Analyzer on its checksum page;
- **Integers and floats (byte order)...** - Byte Analyzer, numbers page;
- **Signal simulation (waveform)...** - Byte Analyzer, signal page, starting from this port's baud rate, data bits, parity and stop bits;
- **Interpret as protocol (Modbus, NMEA...)...** - Byte Analyzer, protocol page;
- **Quick values (click to copy)** - a submenu with the most used results; clicking one copies its value to the clipboard:
  - CRC-16/MODBUS with its two bytes as sent, low byte first, for example `0x0A84 (84 0A)`;
  - CRC-16/CCITT-FALSE, CRC-16/XMODEM, CRC-32, CRC-8, XOR (BCC), SUM8, LRC;
  - for exactly 2 bytes: UInt16, Int16 and Float16 in BE and LE; for 4 bytes: UInt32, Int32 and Float32 in BE, LE and CDAB; for 8 bytes: UInt64, Int64 and Float64 in BE, LE and CDAB;
- **Copy** - copies the selected console text;
- **Copy bytes as hex** - copies the bytes of the message or selection, for example `01 03 00 00`;
- **Select all**, **Clear console**;
- **Export console...** - 8.8;

Copyright © 2026 Fation Coga. All rights reserved.

- **Find...** (Ctrl-F), **Show** (received / sent / notes), **Highlight rules...** - 8.5;
- **Time since the previous message (delta)** - 8.6; **Statistics...** - 8.7;
- **Protocols (info and simulation)...** - opens the Protocols window.

Lines without bytes (notes, file chunk lines) leave the analysis items disabled. The analyzers opened from the console are titled with the source, for example *Byte Analyzer - RX 14:03:21.512 - COM3 - Meter*, or *selection of ...* for a selection.

13. The Byte Analyzer

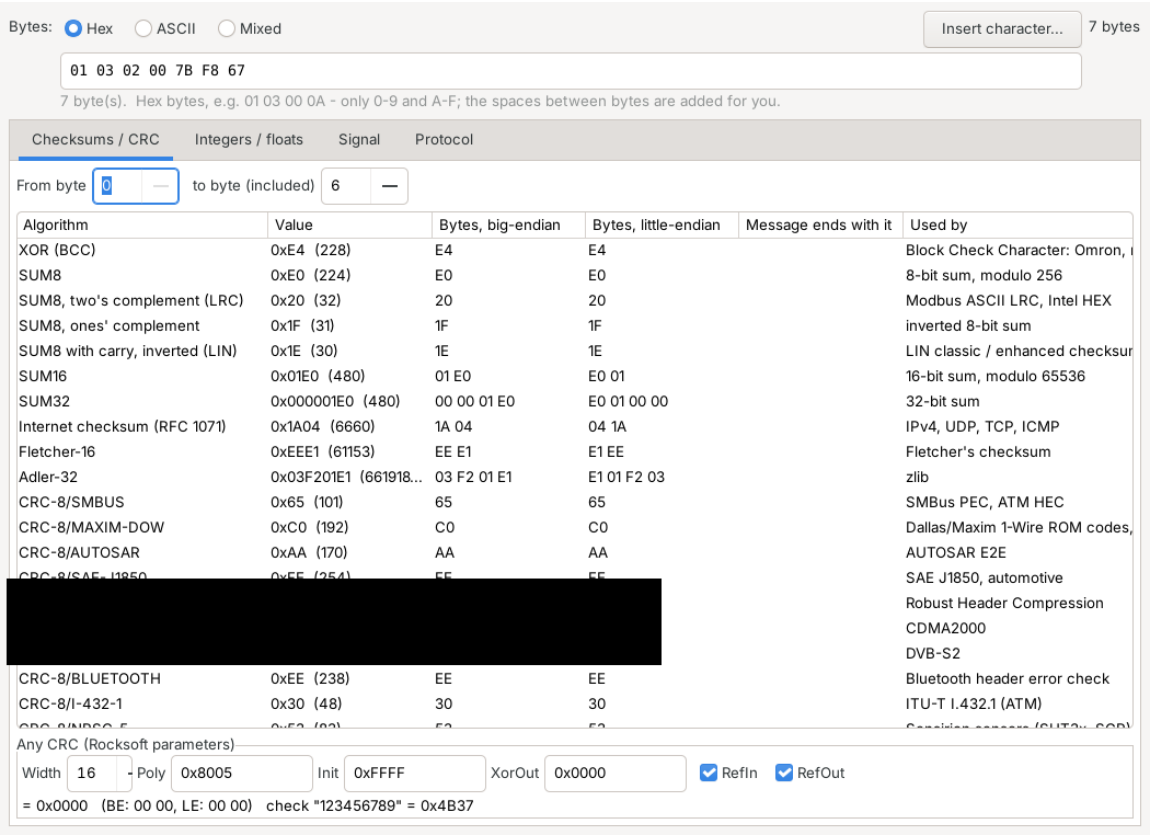
The Byte Analyzer is an independent window for analysing a sequence of bytes. Open it:

- from the console right-click menu (chapter 12), on the bytes of a message or selection, on the page chosen;
- with **Tools > Byte Analyzer...** (Ctrl-B), on the bytes of the send box of the current tab (empty if the send box text is not valid).

Several analyzers can be open at the same time, each with its own bytes; they stay open while you keep working in the main window.

At the top, **Bytes:** shows the bytes in an editable data entry (Hex, ASCII or Mixed, with **Insert character...**) and their number (**N bytes**). Every change is analysed at once on all pages. The four pages are **Checksums / CRC**, **Integers / floats**, **Signal** and **Protocol**.

13.1 Checksums / CRC



The checksum page of the Byte Analyzer

- **From byte** / **to byte (included)**: the range the checksums are calculated on. Bytes are numbered from 0. The end follows the data: when it is at the last byte, it stays at the last byte when bytes are added.

Copyright © 2026 Fation Coga. All rights reserved.

- The list shows about 50 algorithms (52 in this version): 10 sums and checksums and 42 CRCs from CRC-8 to CRC-64, each with its published (Rocksoft) parameters. Columns:
  - **Algorithm** - for example CRC-16/MODBUS, CRC-32/ISO-HDLC, XOR (BCC);
  - **Value** - hexadecimal and decimal;
  - **Bytes, big-endian** and **Bytes, little-endian** - the bytes to append to the message in each order;
  - **Message ends with it** - **yes (BE)** or **yes (LE)**, in green, when the last bytes of the message are already this checksum of the bytes before them (from the first byte of the range): this finds which checksum a device uses;
  - **Used by** - where the algorithm is used (for example *Modbus RTU (sent low byte first)*, *Dallas/Maxim 1-Wire ROM codes, DS18B20*, *Ethernet, ZIP, PNG*).
- Right-click the list: **Copy selected rows** or **Copy all rows** (tab-separated, ready for a spreadsheet).

The sums and checksums: XOR (BCC), SUM8, SUM8 two's complement (LRC), SUM8 ones' complement, SUM8 with carry inverted (LIN), SUM16, SUM32, Internet checksum (RFC 1071), Fletcher-16, Adler-32. The CRCs: ten CRC-8 variants (SMBUS, MAXIM-DOW, AUTOSAR, SAE-J1850, ROHC, CDMA2000, DVB-S2, BLUETOOTH, I-432-1, NRSC-5), CRC-15/CAN, nineteen CRC-16 variants (MODBUS, ARC, IBM-3740, XMODEM, KERMIT, IBM-SDLC, MCRF4XX, GENIBUS, SPI-FUJITSU, USB, MAXIM-DOW, UMTS, CMS, DDS-110, DNP, EN-13757, PROFIBUS, T10-DIF, TELEDISK), CRC-24/OPENPGP, CRC-24/BLE, seven CRC-32 variants (ISO-HDLC, ISCSI, BZIP2, MPEG-2, CKSUM, JAMCRC, AUTOSAR) and three CRC-64 (ECMA-182, XZ, GO-ISO).

Any CRC (Rocksoft parameters) computes a CRC not in the list: **Width** (1 - 64 bits), **Poly**, **Init**, **XorOut** (numbers as 0x1021, 4129 or 0b...), **RefIn** and **RefOut**. It starts with the parameters of CRC-16/MODBUS. The result line shows the value, its bytes big-endian and little-endian, and the *check* value - the CRC of the ASCII text "123456789" - which you can compare with the check value published for the algorithm to confirm the parameters. The line reads = *VALUE (BE: bytes, LE: bytes) check "123456789" = CHECK*; with the starting CRC-16/MODBUS parameters the check is 0x4B37. Invalid parameters: **Write Poly, Init and XorOut as numbers (0x1021, 4129, 0b...).**

13.2 Integers / floats

- **Offset**: the first byte to read (0 = first byte).
- **Byte order**: how the bytes of a number are arranged on the line. With the bytes A B C D as they arrive:
  - **ABCD - big-endian**: most significant byte first (Modbus registers, most network protocols);
  - **DCBA - little-endian**: least significant byte first (PC and most microcontrollers in memory);
  - **BADC - big-endian, bytes swapped**: the two bytes of every 16-bit word swapped;
  - **CDAB - little-endian, words swapped**: the 16-bit words in reverse order, each word big-endian (32-bit values over two Modbus registers, low word first). For 2-byte values CDAB is the same as ABCD.
- **Scale x**: the value is also shown multiplied by this, for example 0.1 for a reading in tenths.

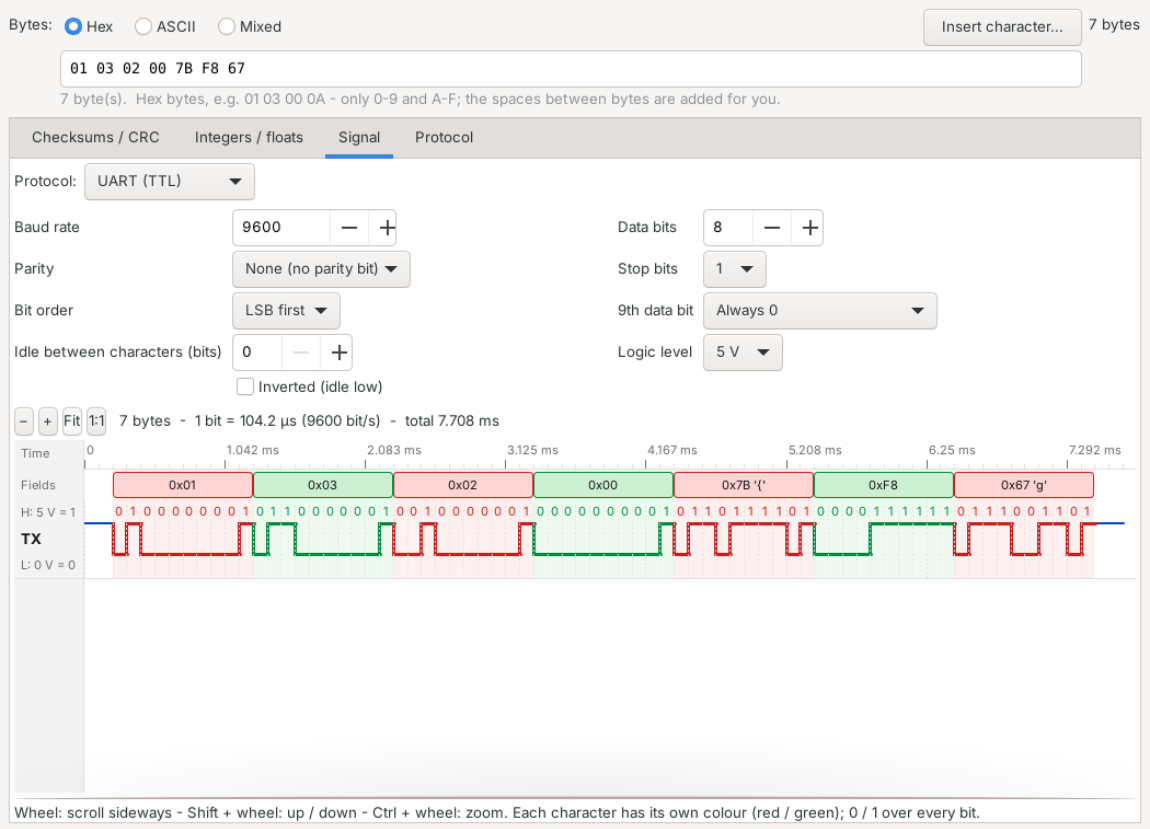
The list shows every type at that offset: Int8, UInt8, Int16, UInt16, Int24, UInt24, Int32, UInt32, Int64, UInt64, Float16 (IEEE 754 half), Float32 (IEEE 754 single), Float64 (IEEE 754 double), BCD 2 bytes, BCD 4 bytes. Columns: **Type**, **Value**, **Scaled** (when the scale is not 1), **Hex (big-endian)** (the bytes rearranged into big-endian order) and **Bytes on the line** (the bytes as they are in the message). A type that does not fit says **needs N bytes from the offset**; a BCD value with a nibble above 9 says **not BCD (a digit above 9)**. Right-click to copy rows.

Value to bytes (and where they are in the message) does the reverse: choose a type (Float32 at first), write a value (decimal, 0x hex, 0b binary; floats as 1.5 or 1e-3) and the line shows the bytes in the byte

Copyright © 2026 Fation Coga. All rights reserved.

order chosen above, for example **Bytes: 3F C0 00 00**, followed by **found at offset 4** (in green) or **not in the message**. An invalid value shows **Not valid** with the reason (empty, not an integer, out of range, negative, not a number, too many digits, only digits 0-9).

13.3 Signal



The Signal page: the bytes of a message as a UART waveform

The Signal page draws the waveform the bytes produce on the wires. **Protocol:** chooses UART (TTL), RS-232, RS-422, RS-485, SPI, I2C, 1-Wire, CAN, LIN, JTAG or NRZ / Manchester; under it are the parameters of that protocol (the same as in the Protocols window, chapter 14.3). The bytes are those at the top of the analyzer.

For UART (TTL), RS-232, RS-422 and RS-485 the analyzer starts from the settings of the port the bytes come from: baud rate, data bits, parity and stop bits. So the waveform shows exactly the bytes of the message as they travelled on the wire. The chosen protocol and the other parameters are remembered. The waveform view is described in chapter 14.4.

13.4 Protocol

Interpret as chooses the decoder:

- **Automatic**: Modbus ASCII if the data starts with `:` and is valid, NMEA 0183 if it starts with $ or ! and is valid, otherwise Modbus RTU if its CRC is correct, otherwise bytes one by one;
- **Modbus RTU**: address, function, data fields, CRC-16/MODBUS (low byte first);
- **Modbus ASCII**: start `:`, address, function, data, LRC, end CR LF;
- **NMEA 0183**: start $ or !, talker and sentence, every field (with names for GGA and RMC sentences), checksum (XOR), end CR LF;
- **Bytes one by one**: each byte in hexadecimal, decimal, binary and as a character.

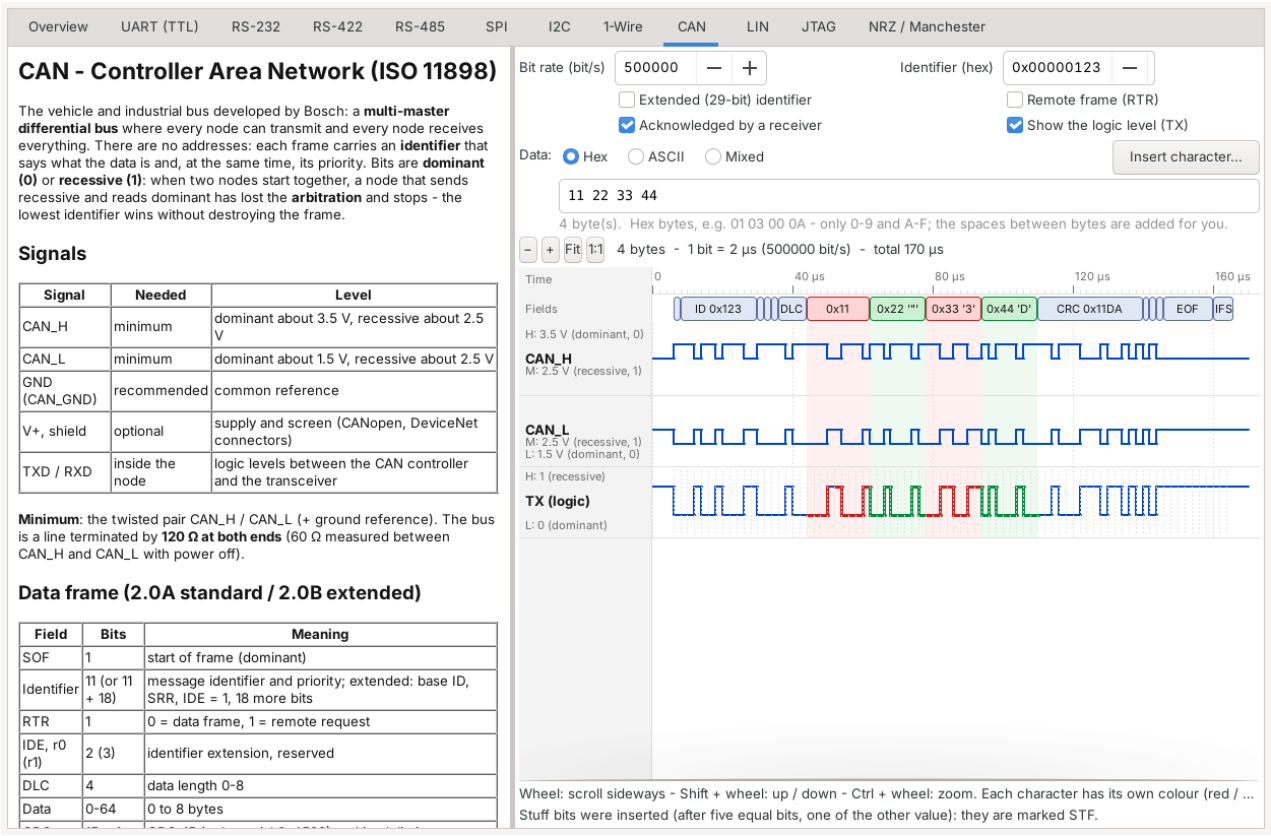
Copyright © 2026 Fation Coga. All rights reserved.

A summary line next to the choice (green when the frame is valid, red when not) is followed by the list **Field**, **Bytes** (the byte range in the message, for example 2-3) and **Value**. Correct checks are green, wrong ones red, with the expected value: for example *CRC-16/MODBUS (low byte first): 0x0A84 correct*, or *wrong, expected 0x0A84 = 84 0A*.

For Modbus the decoder knows the common function codes (01 Read Coils, 02 Read Discrete Inputs, 03 Read Holding Registers, 04 Read Input Registers, 05 Write Single Coil, 06 Write Single Register, 0F Write Multiple Coils, 10 Write Multiple Registers and others), distinguishes requests and responses by their length, shows registers in decimal, hexadecimal and signed, and shows exception responses (function code + 0x80) with their exception code in red.

14. The Protocols window

Tools > Protocols (info and simulation)... (Ctrl-P), or **Protocols (info and simulation)...** in the console right-click menu, opens the window **Standard communication protocols - information and simulation**. It is independent of the main window; opening it again brings the same window to the front.



The Protocols window: the CAN tab, explanation and simulation

14.1 Overview tab

The **Overview** tab (*Communication protocols at a glance*) explains what distinguishes the protocols (asynchronous or synchronous, how many devices share the wires, speed, distance, error checks) and compares them in one table:

Protocol	Clock	Devices	Typical speed	Distance
UART (TTL)	asynchronous	2	9600 - 115200 bit/s	1-2 m
RS-232	asynchronous	2	up to 20 kbit/s (std)	15 m
RS-422	asynchronous	1 driver, 10 receivers	up to 10 Mbit/s	1200 m
RS-485	asynchronous	32 unit loads	up to 10 Mbit/s	1200 m

Copyright © 2026 Fation Coga. All rights reserved.

Protocol	Clock	Devices	Typical speed	Distance
SPI	synchronous	1 master, many slaves	1 - 50 MHz	on the board
I²C	synchronous	112 addresses (7-bit)	100 kbit/s - 5 Mbit/s	< 1 m
1-Wire	time slots	many	15.4 / 125 kbit/s	tens of metres
CAN 2.0	asynchronous	~110 nodes	10 kbit/s - 1 Mbit/s	40 m - 1 km
LIN	asynchronous	up to 16 nodes	1 - 20 kbit/s	40 m
JTAG	synchronous (TCK)	a chain of chips	1 - 50 MHz	on the board

The window's table also gives the signal wires (minimum - maximum), duplex and error check of each protocol, and the tab ends with *How to use this window*.

14.2 Protocol tabs

There is one tab per protocol: **UART (TTL)**, **RS-232**, **RS-422**, **RS-485**, **SPI**, **I2C**, **1-Wire**, **CAN**, **LIN** and **NRZ / Manchester** (line codes). Each tab has two parts, separated by a bar you can drag:

- **Left: the explanation** - what the protocol is for; the signals needed at minimum and at maximum, with direction and meaning; voltage levels; frame structure; speeds and distances; error checks; typical devices and the meaning of the example data.
- **Right: the simulator** - **Data:** (an editable data entry, Hex, ASCII or Mixed), every parameter of the protocol, the zoom bar, the live waveform, the line that describes what is under the mouse, and remarks.

Every tab starts with example data: UART `48 69` ("Hi"), RS-232 `4F 4B 0D` ("OK" + CR), RS-422 `55 AA`, RS-485 `01 03 00 00 00 01 84 0A` (Modbus RTU: read one holding register of slave 1), SPI `9F 00 00 00` (JEDEC "read identification" of a SPI flash), I²C `10 A5`, 1-Wire `CC 44` (Skip ROM, Convert T of a DS18B20), CAN `11 22 33 44`, LIN `55 93 E5`, line codes `4D 3C`. Change the data or any parameter and the waveform follows at once. Parameters are remembered between sessions.

14.3 Simulator parameters

Hexadecimal parameters (addresses, identifiers) are shown and typed in hex. Hovering a parameter shows a hint where one exists.

UART (TTL)

Parameter	Values (default)
Baud rate	50 - 12000000 (9600)
Data bits	5 - 9 (8)
Parity	None (no parity bit), Odd, Even, Mark (always 1), Space (always 0)
Stop bits	1, 1.5, 2
Bit order	LSB first, MSB first
9th data bit	Always 0, Always 1, 1 on the first byte (address) - with 9 data bits: the 9th bit marks an address (multiprocessor / multidrop mode)
Idle between characters (bits)	0 - 50 (0 = back to back)
Logic level	5 V, 3.3 V, 1.8 V
Inverted (idle low)	idle low and 1 = 0 V, as some opto-couplers and level shifters deliver

RS-232 and **RS-422**: the first seven UART parameters (Baud rate to Idle between characters). **RS-485**: the same plus **Show DE (driver enable)** (on): the transceiver's driver enable, high while this node drives the bus (half duplex).

SPI

Copyright © 2026 Fation Coga. All rights reserved.

Parameter	Values (default)
Clock (Hz)	1000 - 200000000 (1000000)
Mode	Mode 0 (CPOL 0, CPHA 0) to Mode 3 (CPOL 1, CPHA 1)
Bits per word	8, 16, 32
Bit order	MSB first, LSB first
Data lines	1 - standard (MOSI / MISO), 2 - dual (IO0-IO1), 4 - quad (IO0-IO3)
MISO answers	Nothing (high impedance), 0xFF (idle slave), The previous word (shift register), The same as MOSI (loopback)
Slaves (CS lines)	1 - 8 (1)
Selected slave	1 - 8 (1)
CS released between words	off - as some ADCs and DACs require
CS active high	off (chip select active low)

I2C

Parameter	Values (default)
Speed	Standard-mode, 100 kbit/s; Fast-mode, 400 kbit/s; Fast-mode Plus, 1 Mbit/s; High-speed mode, 3.4 Mbit/s; Ultra Fast-mode, 5 Mbit/s (write only)
Address (hex)	0 - 3FF (50)
10-bit address	off
Direction	Write (master to slave), Read (slave to master)
Register bytes	0 - 2 (0): register written first; when reading, a repeated start follows
Register (hex)	0 - FFFF (0)
Slave acknowledges	on; off = nobody answers the address (NACK) and the master stops
NACK after the last byte read	on

1-Wire: **Speed** (Standard (~15.4 kbit/s), Overdrive (~125 kbit/s)), **Reset pulse first** (on), **A device answers (presence)** (on), **Who sends** (The master writes, The slave answers (read slots)).

CAN

Parameter	Values (default)
Bit rate (bit/s)	10000 - 1000000 (500000)
Identifier (hex)	0 - 1FFFFFFF (123)
Extended (29-bit) identifier	off
Remote frame (RTR)	off: a request for data, empty data field, DLC tells how many bytes are asked
Acknowledged by a receiver	on; off = the ACK slot stays recessive (an error)
Show the logic level (TX)	on

LIN: **Baud rate** (1000 - 20000, 19200), **Frame ID (hex, 0-3F)** (10), **Enhanced checksum (LIN 2.x)** (on; the checksum also covers the protected identifier, not for the diagnostic frames 3C / 3D), **Break length (bits)** (13 - 26, 13), **With response (data + checksum)** (on; off = header only, the master's request).

JTAG: **Clock (Hz)** (1000 - 100000000, 1000000), **Scan** (Data register only (DR scan), Instruction, then data (IR + DR scan); default the second), **Instruction register bits** (2 - 32, 4), **Instruction (hex)** (0xE), **Reset the TAP first (TMS = 1 x 5)** (on), **TDO answers** (BYPASS: TDI one clock later, All ones, The same as TDI (loopback)), **Show TRST* (optional reset line)** (off). The lanes are TRST* (optional), TCK, TMS, TDI and TDO (dashed while high impedance); the Fields row names the TAP state of every clock (Test-Logic-Reset, Run-Test/Idle, Select-DR-Scan, Capture-IR, Shift-IR, Update-IR...), then the instruction and every data byte, shifted least significant bit first.

NRZ / Manchester: **Bit rate (bit/s)** (1 - 100000000, 1000), **Line code** (NRZ-L, NRZ-I (change on 1), Manchester IEEE 802.3 (1 = low to high), Manchester G.E. Thomas (1 = high to low), Differential Manchester (change on 0), RZ (return to zero), Bipolar AMI; default Manchester IEEE 802.3), **Bit order**

Copyright © 2026 Fation Coga. All rights reserved.

(MSB first, LSB first), **Show the clock** (on).

14.4 The waveform view

The same view is used in the Protocols window and on the Signal page of the Byte Analyzer.

- **The bar above the view**: the minus button (zoom out), **+** (zoom in), **Fit** (show the whole transmission) and **1:1** (normal zoom: every bit readable), followed by the summary, for example *2 bytes - 1 bit = 104.2 µs (9600 bit/s) - total 2.5 ms*.
- **Time**: the time ruler at the top, with a tick for every bit and a time label where there is room.
- **Fields**: one box per character, alternately red and green, with its value (for example *0x48 'H'*), and protocol fields in blue (for example *SOF*, *ID 0x123*, *DLC 4*, *CRC 0x1A2B*, *Address 0x50 W*, *Sync 0x55*). Behind the wires, bands of the same red and green show where each character starts and ends.
- **One row per wire**, named on the left (for example TX, TxD, TX+ (Y), B (+), SCLK, MOSI, MISO, CS, SDA, SCL, DQ, CAN_H, CAN_L, LIN, Clock). Under the name, **H:**, **L:** and, where used, **M:** tell what the high, low and middle level mean on that wire (for example *H: 3.5 V (dominant, 0)*, *M: 2.5 V (recessive, 1)*). The middle level is drawn for recessive CAN, the zero of AMI and - dashed - for a line that is released or not driven (high impedance).
- Over every bit its value **0** or **1** is written in the colour of its character; a stuff bit is written as `0*` or `1*`.
- Under the trace, the role of protocol bits:

Tag	Meaning
ST	start bit (UART), START condition (I²C), SOF (CAN)
SP	stop bit (UART), STOP condition (I²C)
PAR	parity bit
ACK / NAK	acknowledge / not acknowledge (NAK in red)
STF	stuff bit (CAN)
R/W	read / write bit (I²C)
BRK	break (LIN)
DEL	delimiter (CAN, LIN)
Sr	repeated start (I²C)
RST, PD	reset pulse, presence pulse (1-Wire)
ID, CTL, DLC, CRC	identifier, control bits, data length, CRC
EOF, IFS	end of frame, interframe space (CAN)

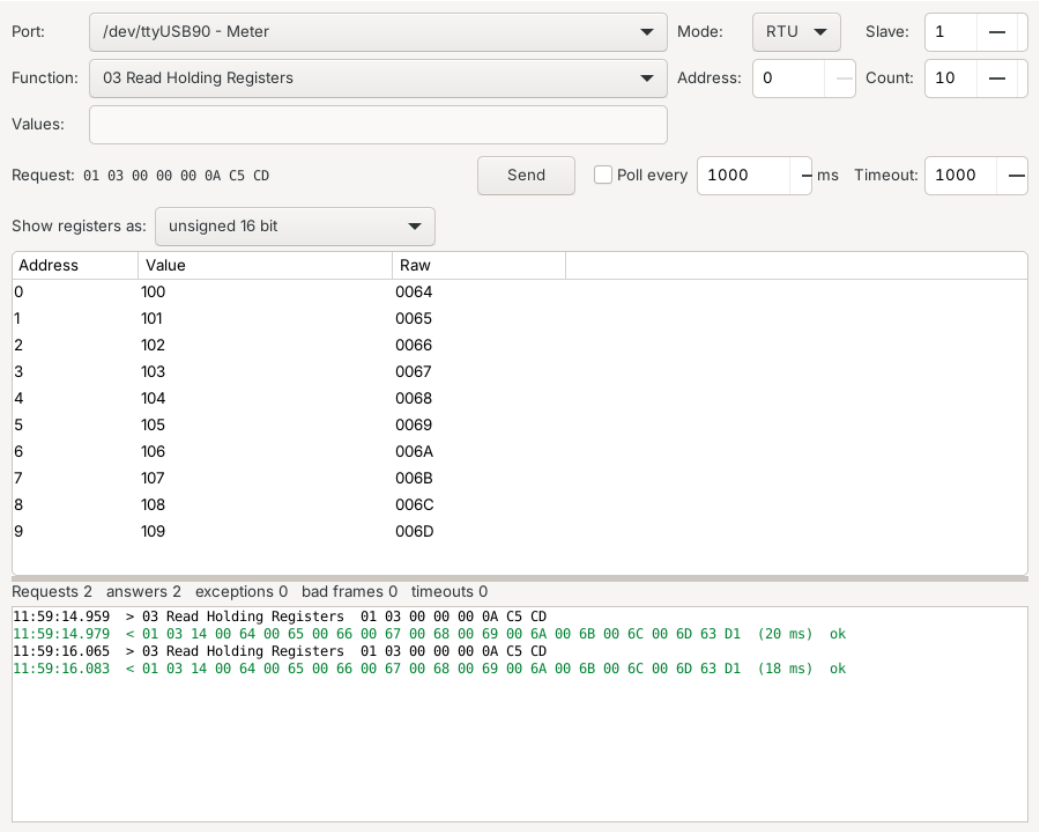
Moving around. The mouse wheel scrolls sideways; Shift + wheel scrolls up and down; Ctrl + wheel zooms around the mouse (the time under the mouse stays under it). The scroll bars work too.

Reading. The line under the view describes the point under the mouse: time, field, wire, level and its meaning, the bit with its role and its position in the byte and its start and end time - for example *t = 1.25 ms | 0x48 'H' | TX: high (5 V = 1) | Data bit 3 of byte 0 = 1 [1.146 ms - 1.25 ms]*. Without the mouse on the view it shows the hint **Wheel: scroll sideways - Shift + wheel: up / down - Ctrl + wheel: zoom. Each character has its own colour (red / green); 0 / 1 over every bit.**

Remarks. Under it, remarks explain what the simulation did with your settings, for example *Stuff bits were inserted (after five equal bits, one of the other value): they are marked STF.*, *CAN 2.0 carries at most 8 data bytes: only the first 8 are sent (CAN FD carries up to 64).*, *A LIN frame carries at most 8 data bytes: only the first 8 are sent.*, *A 7-bit address goes up to 0x7F: the higher bits are ignored.*, *The slave did not acknowledge: the master stops the transfer.*, *No presence pulse: no device answered the reset.*, *The last word is completed with zero bytes.*, *Some bytes do not fit in the data bits: only their low bits are sent.*

Copyright © 2026 Fation Coga. All rights reserved.

15. The Modbus Master



The Modbus Master window reading ten holding registers

Tools > Modbus Master... (Ctrl-M) reads and writes the coils and registers of a Modbus RTU or ASCII slave through the port of one of the tabs.

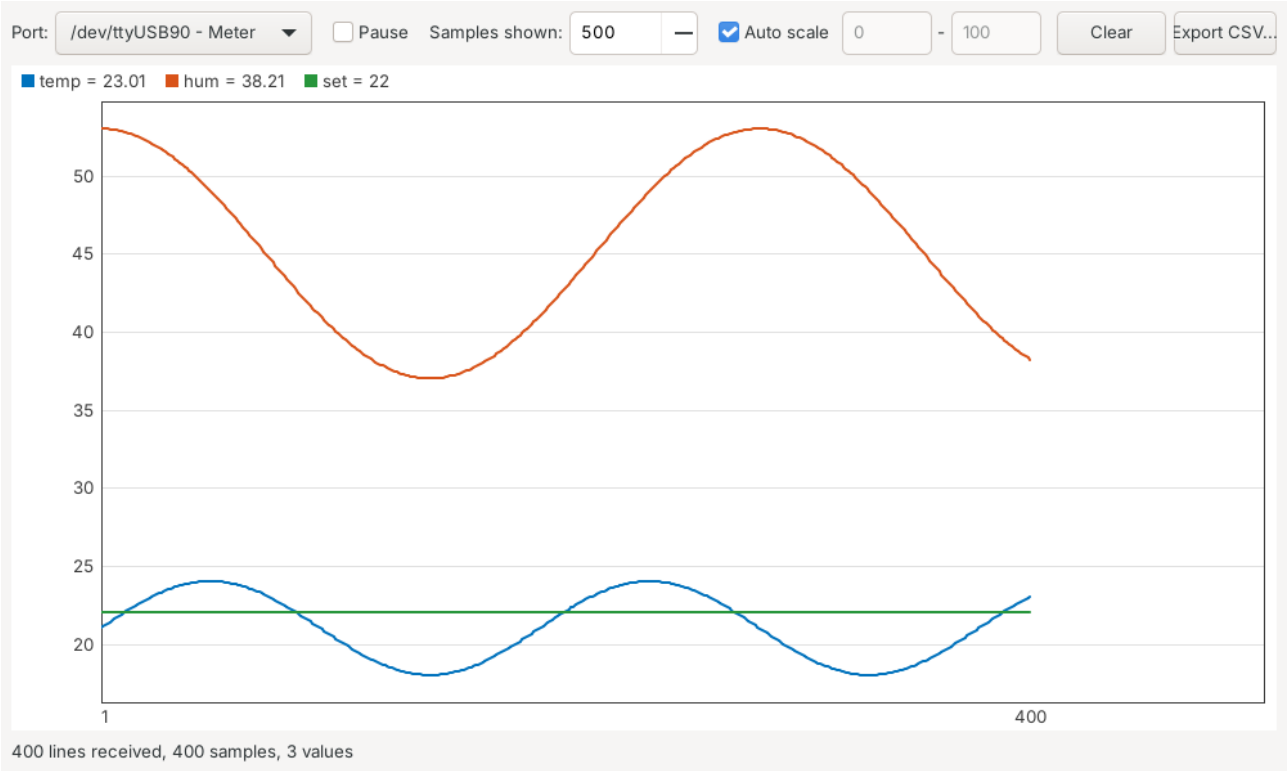
- **Port:** the tab whose port the requests go through - it must be open; its console shows the requests as **TX Modbus** and the answers as **RX**.
- **Mode: RTU** (binary, CRC-16) or **ASCII** (**:** hex text, LRC, CR LF).
- **Slave:** 1 - 247; 0 = broadcast (writes only, nobody answers).
- **Function: 01 Read Coils**, **02 Read Discrete Inputs**, **03 Read Holding Registers**, **04 Read Input Registers**, **05 Write Single Coil**, **06 Write Single Register**, **15 Write Multiple Coils**, **16 Write Multiple Registers**.
- **Address:** the protocol address, from 0 (holding register 40001 is 0); **Count:** 1 - 125 registers or 1 - 2000 coils for the reads.
- **Values:** for the writes: numbers, decimal or `0x` hex, separated by spaces (coils: 0 / 1).
- **Request:** the frame as it will be sent, or in red why it cannot be.

Send sends it once; **Poll every N ms** sends it again and again (the next one after the answer or the timeout and the interval). **Timeout:** how long to wait for the answer.

The answer is checked - CRC / LRC, slave, function, length - and Modbus exceptions are named (for example *exception 2: Illegal data address*). The values read are shown in the table (**Address**, **Value**, **Raw**) as chosen in **Show registers as:** unsigned / signed 16 bit, hex, binary, or 32-bit unsigned, signed and float with the high or the low word first. The log keeps every request (>) and answer (<) with its time and round-trip time, and the counters show requests, answers, exceptions, bad frames and timeouts. The Modbus master needs the license feature *Sending*.

Copyright © 2026 Fation Coga. All rights reserved.

16. The Serial Plotter



The Serial Plotter with three values

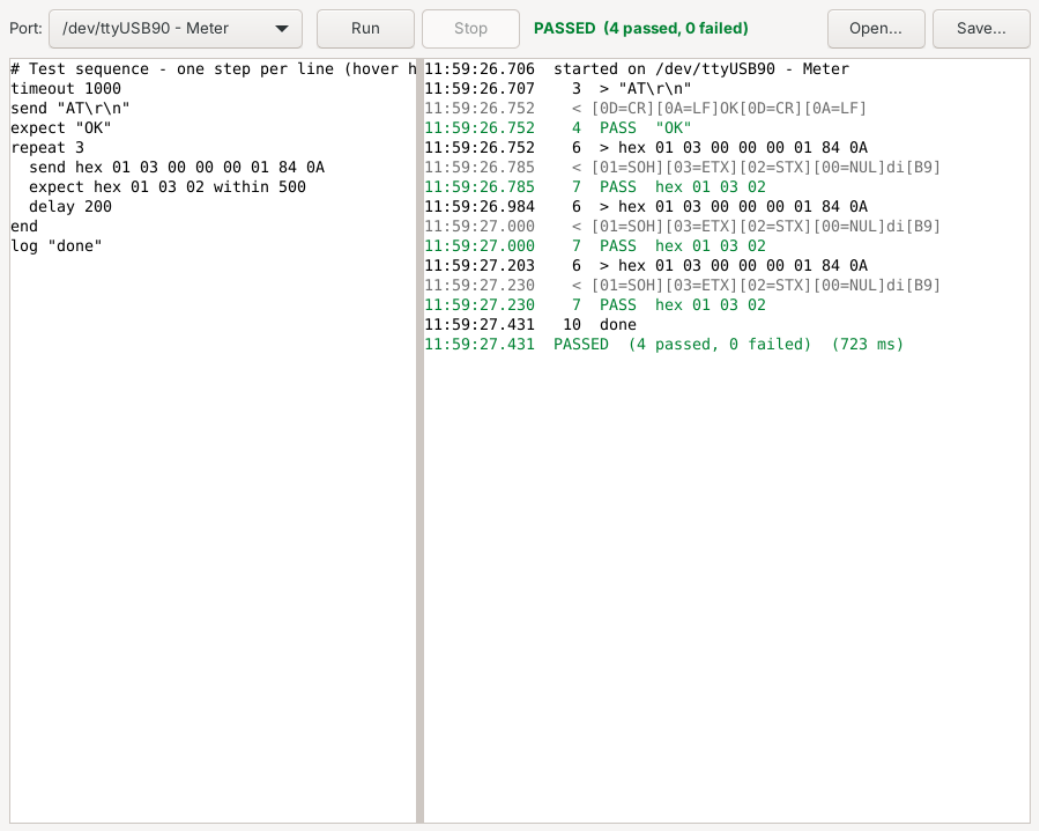
Tools > Serial Plotter... (Ctrl-L) draws the numbers a device prints, as the Arduino IDE's plotter does. Every text line (ended by LF, CR or CR LF) is one sample; its values are separated by commas, semicolons, tabs or spaces, plain (`21.5,40`) or named (`temp:21.5 hum=40`); words that are not numbers are skipped (`T= 21.5 C` gives 21.5 named T). Unnamed values are called 1, 2, 3...

- **Port:** the tab to listen to (its console keeps working as usual).
- **Pause** freezes the picture; the samples are still collected.
- **Samples shown:** the last N samples (10 - 100000).
- **Auto scale**, or the bottom and top of a fixed scale.
- **Clear**, **Export CSV...** (sample, time and every value).

One colour per value (up to 16), a legend with the last values on top, the scale on the left and the numbers of the first and last sample under the plot.

17. Test Sequences

Copyright © 2026 Fation Coga. All rights reserved.



A test sequence that passed

Tools > Test Sequences... (Ctrl-T) runs a script of steps against a device through the port of a tab; every expectation passes or fails. One step per line (hover over the script for this list):

Step	Meaning
`send "AT\r\n"`	sends text; escapes `\r \n \t \0 \\ \"` and `\xHH` for any byte
`send hex 01 03 00 00`	sends hex bytes
`send mixed AT//0x0D`	sends bytes in the Mixed notation
`expect "OK"`	waits until these bytes arrive after the last send: pass, or fail after the timeout
`expect hex 01 03 within 200`	... waiting at most 200 ms
`timeout 1000`	the default wait of expect (ms; 1000 at the start)
`delay 250`	waits 250 ms
`repeat 10` ... `end`	repeats the steps between (0 = until stopped); may nest
`log "text"`	writes text to the log
`onfail continue` / `onfail stop`	after a failed expect: go on, or end the run (default)
`# ...`	a comment

Run checks the script (errors are listed with their line) and runs it; **Stop** ends it. The log shows every step with its line number, what was sent (>), what was received (<), **PASS** in green and **FAIL** in red with what had arrived. The result is **PASSED (n passed, 0 failed)** or **FAILED (...)**. The script is kept between sessions; **Open...** and **Save...** read and write it as a file (`*.spmseq`). Test sequences need the license feature *Sending*.

18. Profiles

A profile holds the setup of the ports: every tab's port settings, control lines, automation, send box, receive options and console options, the saved commands and the highlight rules - not the language, the theme, the log level, the event database limits or the license. **File > Export Profile...** saves it to a file

Copyright © 2026 Fation Coga. All rights reserved.

(`*.spmprofile`); **File > Import Profile...** replaces the current setup with a file's (after a question; open ports are closed) and rebuilds the tabs. Use it to move a setup to another PC or to keep one per project.

19. Licensing

SerialPortManager is licensed per PC. This chapter describes licensing from the user's point of view.

19.1 Trial, licensed and not licensed

State	What works
Trial (14 days from the first start on the PC)	every feature
Licensed	the features of the license
Not licensed (trial over, no valid license)	ports open, everything received is shown and logged; Byte Analyzer and Protocols window

The features a license can include:

Feature	Unlocks
Sending	sending from the send box, saved commands, replay, the Modbus master, test sequences
Auto replies	automatic replies to received messages
Periodic messages	periodic messages
File transfer	Send File and Catch File
Forwarding to another port	Forward received to

Licenses are issued in editions: **professional** includes every feature, including features of future versions; **standard** includes sending; other combinations can be issued.

Using a feature that is not available opens the question *"Sending needs a license. ... Open the License window?"*, with the reason (for example *Your license (edition: standard) does not include it.* or *The trial period is over.*).

19.2 The License window

Help > License... (Ctrl-K) opens the **License** window:

- A coloured LED and the state: **Licensed** (green; yellow in the last 14 days before expiry), **Trial version - N day(s) left** (blue) or **Not licensed** (red), with the reason when a license is present but refused.
- The details of the license: **Licensed to:**, **E-mail:**, **Edition:**, **License ID:**, **Valid on:** (**this PC**, **N PCs, this one included**, or **any PC (site license)**), **Issued:**, **Expires:** (a date with the days left, or **never (perpetual)**), **Updates until:** (a date, or **all versions**), **Notes:**, **File:**.
- **Features:** every feature, **available** or **not included**.
- **This PC's unique ID (UID):** the identifier of this PC, with **Copy**.
- The buttons **Save Request...**, **Load License...**, **Paste License**, **Remove License**, **Open Folder** and **Close**.

19.3 Getting a license

1. Open **Help > License...**.
2. Send to the license issuer either the UID (click **Copy** and paste it into an e-mail) or the request file: **Save Request...** saves it where you want (for example to attach it to an e-mail); a copy, license-request.txt, is always in the data folder (**Open Folder**). The request contains the product, the UID, the computer name, the version and - for a renewal - the current license ID.

Copyright © 2026 Fation Coga. All rights reserved.

3. You receive a license file (for example license-SERIALPM-....lic).
4. Install it with **Load License...** (choose the file), **Paste License** (when you received the license as text, copy it and click), or by dropping the file on the License window. You can also copy it as license.lic into the data folder (or next to the exe) before starting the program.

The program checks the license at once: **License installed - licensed to NAME.**, or **This license cannot be used on this PC:** with the reason. The features become available immediately (auto replies and periodic messages start if their switches are on).

Remove License asks **Remove the installed license? It is kept as license.lic.bak in the data folder.**

19.4 What a license checks

- **The signature**: a license is digitally signed by the issuer. Any change to the file makes it invalid (**The license was not issued by the license owner, or it was changed after it was issued.**).
- **The product**: a license for another product is refused.
- **The PC**: the UID is derived from the Windows installation of the PC. Reinstalling Windows changes it (a new license is needed); changing hardware does not. A license may name several PCs, or any PC (site license).
- **Expiry**: a time-limited license works until its expiry date; the status bar warns in the last 14 days.
- **Updates until**: the license covers the versions released until that date; a newer version needs a renewal (**The license covers versions released until ...; this version was released on .... Keep using an older version or renew the license.**).
- **Revocation**: a revoked license is refused by newer versions.
- **The clock**: if the PC date is set back by more than 2 days compared with a date already seen, the trial and time-limited licenses are suspended until the date is correct. Perpetual licenses are never affected.
- **The trial state** is stored twice, with check values; editing it ends the trial (**The trial information was changed - the trial is over.**).

The license state is checked again once a day while the program runs.

20. Settings

20.1 Theme

Settings > Theme... offers **Light** and **Dark** (light is the default). After choosing, the program asks how to apply it:

- **Restart now**: save and restart the program automatically;
- **Use now**: save and use it right away; parts drawn by the system (title bars, scroll bars) change at the next start;
- **Save for next start**: keep working, apply at the next start;
- **Cancel**: discard the choice.

In the dark theme, status colours are chosen to stay readable (for example green and red in the analyzers and the waveform).

20.2 Language

Settings > Language... lists **English** and the languages whose file is in the data folder (**Italiano**). The same question as for the theme follows. With **Use now**, menus and windows opened from then on use the

Copyright © 2026 Fation Coga. All rights reserved.

new language; the other labels of the main window change at the next start. If a language file cannot be loaded, the current language is kept.

20.3 Log level

Settings > Log Level... - **Write to the log messages up to:** None, Error, Warning, Info (default), Debug, Verbose, Trace. Higher levels write more detail to the Application log and to log.txt; use Debug or above only to investigate a problem.

20.4 Open Data Folder

Settings > Open Data Folder opens the data folder (chapter 2.3) in Explorer.

20.5 Event Database

Settings > Event Database... shows the file events.db (chapter 2.3), the number of events and its size, and sets:

- **Record the serial traffic (every message received and sent)** - on by default; off, only the program's own events (start, ports, license, files) are kept;
- **Keep events for (days)** - default 30, **0 = no limit**: older events are deleted;
- **Largest size (MB)** - default 200, **0 = no limit**: the oldest events are deleted until the database fits.

The limits are applied at the start, every 10 minutes and when the window closes with **OK**. **Export to CSV...** saves every event (time, kind, text); **Compact now** gives the space of deleted events back to the disk; **Delete all...** empties the database after a question.

21. Help menu

- **User Manual** (F1): this manual as a PDF, in the language in use.
- **Quick Guide**: the short guide as a PDF.
- **License...** (Ctrl-K): chapter 19.
- **Version Info...**: the **Version Information** window with version, build number and date, build type (**Development (Debug)**, **Release candidate** or **Final release**), source (git), the data folder, and the version history (newest first).
- **About...**: a window with tabs - **About** (name, version and build date, description, author, contact, project page, copyright, license and its state, what the program does), **Developer** (e-mail, GitHub pages, **Report a problem by e-mail...**, which opens an e-mail with the details of this PC filled in), **Dependencies** (every library with its version, licence and use; names and licences are links), **Ports** (the serial ports on this PC, what each device is, its hardware id and the tab using it), **License** (state, features, this PC's UID, **License window...**), **Changes** (the version history) and **System** (versions, folders, database, ports, license, UID - **Copy to clipboard** for a support request).

22. Troubleshooting

Problem	What to do
Cannot open COM3: ...	The port is used by another program (close it), unplugged, or the name is wrong. Click **Refresh** in the port settings and choose again.
The port is not in the list	Plug in the adapter, install its driver, click **Refresh**. Virtual and Bluetooth ports appear only when their driver is active.

Copyright © 2026 Fation Coga. All rights reserved.

Problem	What to do
How many serial ports to control cannot be raised	The maximum is the number of ports present now (at most 32). Connect the adapters first.
Port already used	The port belongs to another tab: choose **Detach and use here** or another port.
Nothing is received	Check baud rate, data bits, parity and stop bits on both sides; cross TX and RX (TX of one side to RX of the other); connect GND; with **Hardware (RTS/CTS)** check that the other side drives CTS, or choose **None**. For RS-485 swap A and B if nothing arrives.
Received text is garbage	Wrong baud rate or parity, or wrong levels (RS-232 directly on a TTL input). The Signal page shows how the bytes should look on the line.
Messages split or merged	Increase or decrease the **Inter-byte timeout (ms)** (chapter 6), or cut them by delimiter, length or Modbus timing in the **Receive** options (8.4).
Write failed (...)	The write timeout passed: flow control blocking, the adapter removed, or a write timeout too short.
The port stopped working	A USB adapter was unplugged: the port is opened again when it comes back (6.3), or close and open it again.
Send is disabled	Open the port; wait for a file transfer to end.
Too many messages waiting	Periodic messages faster than the line can carry them: make the period longer or the baud rate higher.
Auto replies do not reply	Is the **Auto replies** switch on, the rule ticked, the license including Auto replies? Is the pattern exactly the bytes received (check the notation)? Auto replies pause while a file is sent.
A periodic message stopped	Its text cannot be built: edit and fix it.
File sending fails	Read the reason in the console (for example *no confirmation for chunk 4 (expected: ...)*): check the confirmation pattern, the answer timeout, the baud rate and that both sides use the same commands and chunk / group size.
Catch File refuses the options	Choose an output file and either an end marker or a silence time.
The manual was not found	The PDF manuals are missing from the **manuals** folder next to the exe: copy them there.
A license is refused	Read the reason in the License window: other PC (send the UID of this PC), expired, updates not covered (use an older version or renew), clock set back (correct the date).
The trial ended early	The PC date was set back, or the trial information was edited.
A control line does not change	Virtual and some USB ports have no modem lines (the console note is red); with hardware flow control RTS belongs to the driver.
The Modbus master gets no answer	Check slave address, mode (RTU / ASCII), baud rate and parity; with RS-485 the adapter must switch direction by itself. A timeout with some bytes received means a wrong format or a too short timeout.
The plotter shows nothing	The device must end its lines (LF or CR) and print numbers; the status line counts the lines received.
A test sequence fails at once	**expect** looks only at what arrives after the last **send**; make the timeout longer (`within`, `timeout`).

If a problem remains, set **Settings > Log Level...** to Debug, repeat the operation and send log.txt (and, if useful, events.db) from the data folder to support.

23. Keyboard and mouse shortcuts

Copyright © 2026 Fation Coga. All rights reserved.

Keys / mouse	Where	Action
Alt-X	main window	Exit
Ctrl-B	main window	Byte Analyzer on the send box bytes
Ctrl-P	main window	Protocols window
F1	main window	User Manual
Ctrl-K	main window	License window
Ctrl-M	main window	Modbus Master
Ctrl-L	main window	Serial Plotter
Ctrl-T	main window	Test Sequences
Enter	Send box	send
Up / Down	Send box	messages sent before
Ctrl-F	console	find bar
F3 / Shift-F3	console	next / previous match
Enter / Shift-Enter, Esc	find bar	next / previous, close
right-click	console	analysis menu
context-menu key	console	analysis menu for the selection
double-click	lists of auto replies, periodic messages, incrementing values	edit the item
double-click	Insert Character table	add the character and close
right-click	Insert Character table	add the character and keep the table open
right-click	Byte Analyzer lists	copy rows
wheel	waveform	scroll sideways
Shift + wheel	waveform	scroll up / down
Ctrl + wheel	waveform	zoom around the mouse
Esc	dialogs	close / cancel

24. Glossary

Term	Meaning
8N1	8 data bits, no parity, 1 stop bit: 10 bit times per byte
ACK / NACK	acknowledge / not acknowledge
ASCII	the 7-bit character code; printable characters are 20-7E hex
Baud rate	symbols (here bits) per second on the line
BCC	Block Check Character, an XOR of the bytes
BCD	Binary Coded Decimal: two decimal digits per byte
Big-endian / little-endian	most / least significant byte first
Break	the TX line held at 0 for longer than a character: a reset or frame marker
Burst mode	file sending that waits for a confirmation after every chunk
Capture	the traffic of a port written to a file while it happens
Checksum	a value computed from the bytes to detect errors
Chunk / group	the piece of a file sent / answered at a time
CPOL / CPHA	SPI clock polarity and phase
CRC	Cyclic Redundancy Check, a strong checksum
CTS / DSR / DCD / RI	modem inputs: clear to send, data set ready, data carrier detect, ring indicator
Dominant / recessive	CAN and LIN bus levels: dominant (0) overrides recessive (1)
End marker	bytes that mark the end of a file
Flow control	a way for the receiver to pause the sender (XON/XOFF or RTS/CTS)

Copyright © 2026 Fation Coga. All rights reserved.

Term	Meaning
Framing	how a stream of bytes is cut into messages (pause, delimiter, length...)
Inter-byte timeout	the silence that ends a received message
LRC	Longitudinal Redundancy Check: two's complement of the sum
Parity	an extra bit that makes the number of ones odd or even
Poll	a request sent again and again at an interval
Prefix	bytes sent before a file to announce it
Profile	the whole setup of the ports in one file
Recovery	continuing an interrupted file transfer where the receiver stopped
Rocksoft parameters	Width, Poly, Init, RefIn, RefOut, XorOut: the published description of a CRC
RTS / DTR	modem outputs: request to send, data terminal ready
RX / TX	received / transmitted
Site license	a license valid on any PC of the licensee
Slot	a tab of the main window, with its own port
Stuff bit	a bit of the opposite value inserted after five equal bits (CAN)
UART	the asynchronous serial interface (start bit, data, parity, stop)
UID	the unique identifier of a PC used by the license

25. Legal notices

Copyright (C) 2026 Fation Coga. All rights reserved.

SerialPortManager and this manual are the property of Fation Coga. SerialPortManager is proprietary software, licensed per PC under the terms of its License Agreement (LICENSE.md): it is licensed, not sold. No part of this manual may be reproduced, distributed or translated, in any form or by any means, without the written permission of the copyright owner.

The software is provided "as is", without warranty of any kind, to the extent permitted by law; see the License Agreement for the disclaimer of warranties and the limitation of liability. SerialPortManager communicates with external equipment: you are responsible for verifying its suitability and for the safe operation of any equipment connected to it.

Third-party components. SerialPortManager includes or links the following components, each under its own licence; their notices are in the file THIRD-PARTY-NOTICES.md shipped next to the exe:

- wxWidgets - wxWindows Library Licence 3.1, by Julian Smart, Vadim Zeitlin, Stefan Csomor, Robert Roebling and other members of the wxWidgets team (https://www.wxwidgets.org/);
- SQLite - public domain (https://www.sqlite.org/);
- serial - the CogaF fork of wjwwood/serial, MIT licence, Copyright (c) 2012 William Woodall, John Harrison (https://github.com/wjwwood/serial);
- TweetNaCl - public domain, by Daniel J. Bernstein, Wesley Janssen, Tanja Lange, Peter Schwabe, Matthew Dempsky and Sjaak Smetsers, used for the Ed25519 signature arithmetic (https://tweetnacl.cr.yp.to/).

Trademarks. Microsoft and Windows are trademarks of Microsoft Corporation. wxWidgets is the name of the wxWidgets project, by the wxWidgets team. SQLite is a registered trademark of Hipp, Wyrick & Company, Inc. Modbus is a registered trademark of Schneider Electric USA, Inc. I²C is a trademark of NXP B.V. 1-Wire and iButton are registered trademarks of Maxim Integrated Products, Inc., now part of Analog Devices, Inc. CAN is standardised in ISO 11898 and was developed by Robert Bosch GmbH. LIN is standardised in ISO 17987. NMEA 0183 is a standard of the National Marine Electronics Association. JTAG is standardised in IEEE 1149.1. Arm and CoreSight are trademarks of Arm Limited. All other names

and trademarks belong to their respective owners and are used only to identify the products and protocols; their use does not imply any affiliation with or endorsement by them.

Copyright © 2026 Fation Coga. All rights reserved.